RELATIONAL DATABASE PROJECT · MYSQL 8

Consulting Delivery Performance Analytics

A single-table MySQL case study on consulting delivery data — 700 project records for 120 consultants across four departments and three client tiers — answered with five analytical questions built from CTEs, window functions and scenario modelling: productivity benchmarking, quality trajectories, counterfactual staffing, promotion alignment and executive risk prioritisation.

700	120	176	+0.98%
DELIVERY RECORDS	CONSULTANTS	PROJECTS	REALLOCATION UPLIFT

124.08M	282.90M	82.1	10.2%
TOTAL CONTRIBUTION	RECOGNISED REVENUE	AVG QUALITY SCORE	DEPT AVG REWORK RATE

DATABASE ENGINE	MySQL 8.0	SCHEMA	case_study_2
SOURCE TABLE	consulting_delivery_performance	ANALYTICAL QUESTIONS	5 questions · 27 steps
CONCEPTS COVERED	CTEs, NTILE(), RANK(), LAG(), ROW_NUMBER(), running AVG() OVER, CASE, CROSS JOIN, scenario UNION ALL		

WHAT THIS DOCUMENT CONTAINS

The setup statements and every analytical query from the original script, reproduced in full and grouped into five questions and twenty-seven steps; the executed result sets as captured from MySQL; charts that summarise the department and segment findings; and a code-review section that records the defects and design limits found while reading the queries against their own output, with a correction for each.

The source dataset itself was not part of the supplied script, so the table is documented from the columns the queries reference. Result sets are the rows visible in the executed output, which for row-level queries is an excerpt rather than the full result.

CONTENTS

Table of Contents

Project Overview 3

Data Model & Setup 5

Productivity Benchmarking 7

Question 1 — Productivity benchmarking under heterogeneous project conditions 7

Performance Trajectory 13

Question 2 — Performance trajectory and sustained underperformance 13

Counterfactual Staffing 21

Question 3 — Counterfactual staffing and resource allocation 21

Compensation & Promotion Alignment 32

Question 4 — Compensation and promotion alignment 32

Executive Decision Under Uncertainty 42

Question 5 — Executive decision under uncertainty 42

Code Review & Corrections 50

Results & Roadmap 53

01
SECTION

Project Overview

One wide fact table, five business questions: where consulting delivery creates contribution, where it leaks, and which conclusions survive a change of benchmark.

The **consulting_delivery_performance** table holds one row per employee–project assignment. Each row carries the hours a consultant billed and did not bill, the rework hours, the revenue recognised and the delivery cost, a quality score, and the consultant's department, job level, status and last promotion date. The five questions move from descriptive benchmarking to counterfactual modelling, so each one builds on the metric definitions of the one before.

Objectives

- **Benchmark fairly.** Compare consultants inside peer groups defined by department, job level, client tier and project complexity, and measure how much rankings move versus the company-wide view.
- **Detect trajectories.** Flag consecutive quality declines and revenue gains that are not matched by quality-adjusted productivity.
- **Model the counterfactual.** Estimate the contribution change from moving 10% of delivery hours from below-median to above-median segments, and test it against alternative populations.
- **Check promotion alignment.** Find high performers without a recent promotion and recent promotees below their level's median quality.
- **Prioritise under uncertainty.** Rank resource-allocation risks and separate robust findings from benchmark-sensitive ones.

Analytical Modules

Q	Theme	Core techniques	Steps
Q1	Productivity benchmarking under heterogeneous project conditions	Utilisation, RANK() per peer group, NTILE(10), median via ROW_NUMBER()	4
Q2	Performance trajectory and sustained underperformance	LAG(), streak groups, running department benchmarks, quality-adjusted productivity	6
Q3	Counterfactual staffing and resource allocation	Median split, donor/recipient pools, 10% reallocation, scenario UNION ALL	6
Q4	Compensation and promotion alignment	Promotion recency, peer medians, relative productivity, status comparison	6
Q5	Executive decision under uncertainty	Rule-based risk segments, benchmark swap, sensitivity bands, priority ranking	5

Technology and Techniques

Area	Applied in this project
Aggregation	SUM, AVG, COUNT, MIN, MAX; ratio metrics always divided through NULLIF(denominator, 0)
Window functions	RANK() OVER (PARTITION BY ...), NTILE(10), LAG(), ROW_NUMBER(), COUNT(*) OVER (), running AVG() with ROWS UNBOUNDED PRECEDING
Medians in MySQL	ROW_NUMBER() with FLOOR((n+1)/2) and FLOOR((n+2)/2) averaged, because MySQL has no MEDIAN() function
Streak detection	Gaps-and-islands: a running SUM of non-decline flags labels each run of consecutive declines
Scenario modelling	CROSS JOIN of a one-row pool CTE; UNION ALL of three scenario populations with PARTITION BY scenario
Date logic	DATEDIFF(CURDATE(), promotion_date) against a 1,095-day (three-year) threshold

Area	Applied in this project
Segmentation	CASE expressions that turn numbers into labels: Donor / Recipient, risk segment, sensitivity band, promotion review flag

02
SECTION

Data Model & Setup

The table the five questions run against, the setup statements, and the metric definitions used throughout.

Setup statements

SETUP — DATABASE AND SOURCE TABLE

```
CREATE DATABASE case_study_2;

USE case_study_2;

SELECT * FROM `consulting_delivery_performance`;

SHOW TABLES;
```

The table is loaded from an external file rather than created in the script, so its definition is inferred from the columns the queries read. Types shown are the ones the operations imply.

Columns referenced by the analytical queries

Column	Inferred type	Used for
record_id	VARCHAR (R00001…)	Row identifier; tie-breaker in every ORDER BY of the trajectory queries
employee_id	VARCHAR (E0001…)	Consultant identifier (120 distinct)
project_id	VARCHAR	Project identifier (176 distinct); sensitivity filter on project hours
department	VARCHAR	Analytics, Operations, Strategy, Technology
job_level	VARCHAR	L1 to L6
client_tier	VARCHAR	Enterprise, Mid-Market, Strategic
project_complexity	INT	Complexity score 1 to 9
project_start_date	DATE	Ordering of projects within an employee
billable_hours / non_billable_hours	DECIMAL	Utilisation and revenue per billable hour
rework_hours	DECIMAL	Rework rate = rework ÷ (billable + non-billable)
recognized_revenue / delivery_cost	DECIMAL	Contribution proxy = revenue − delivery cost
quality_score	DECIMAL	Quality of the delivered work (60 to 100 in the excerpts)
employee_status	VARCHAR	Active or Exited
promotion_date	DATE	Last promotion; basis of promotion recency

Metric definitions

Metric	Definition
Utilisation %	SUM(billable_hours) ÷ SUM(billable_hours + non_billable_hours) × 100
Revenue per billable hour	SUM(recognized_revenue) ÷ SUM(billable_hours)
Contribution proxy	SUM(recognized_revenue) − SUM(delivery_cost); a proxy because indirect cost is not in the table
Rework rate %	SUM(rework_hours) ÷ SUM(billable_hours + non_billable_hours) × 100
Contribution per billable hour	Contribution proxy ÷ SUM(billable_hours); the reallocation yardstick in Q3
Quality-adjusted productivity	Revenue per billable hour × quality score; used in the Q2 trajectory check
Relative productivity	Employee revenue per billable hour ÷ peer-group revenue per billable hour (department and level)

Dataset profile

Measure	Value	Source
Delivery records	700	Sum of department record counts (Q5 Step 1)
Consultants	120	eligible_employees, Q3 Step 6
Projects	176	eligible_projects, Q3 Step 6
Recognised revenue	282,904,868.67	Sum of segment revenue (Q3 Step 1)
Delivery cost	158,828,903.10	Sum of department delivery cost (Q5 Step 1)
Contribution proxy	124,075,965.57	Revenue − cost; equals the sum of department contributions
Employee–department–level–status groups	697 (599 Active, 98 Exited)	Q4 Step 6 employee_count

03
SECTION

Productivity Benchmarking

Question 1: consultant-level utilisation, revenue per billable hour and quality, ranked against peers rather than the whole firm.

Question 1 — Productivity benchmarking under heterogeneous project conditions

Calculate employee-level utilisation, revenue per billable hour, contribution proxy, rework rate and quality score. Rank employees within comparable department, job-level, client-tier and project-complexity groups; find employees in the top 10% for revenue per billable hour but below median quality; and quantify how rankings change between company-wide and peer-group comparison.

Step 1 — Employee-level utilisation and productivity

One row per employee, department and job level with five metrics. Grouping by department and level means a consultant who worked in several departments appears once per department — E0001 appears five times in the excerpt.

QUESTION 1 · STEP 1 — EMPLOYEE-LEVEL UTILISATION AND PRODUCTIVITY

```sql
SELECT
  employee_id,
  department,
  job_level,
  ROUND(
    SUM(billable_hours) * 100.0 /
    NULLIF(SUM(billable_hours + non_billable_hours), 0),
    2
  ) AS utilization_pct,
  ROUND(
    SUM(recognized_revenue) /
    NULLIF(SUM(billable_hours), 0),
    2
  ) AS revenue_per_billable_hour,
  SUM(recognized_revenue) - SUM(delivery_cost)
    AS contribution_proxy,
  ROUND(
    SUM(rework_hours) * 100.0 /
    NULLIF(SUM(billable_hours + non_billable_hours), 0),
    2
  ) AS rework_rate_pct,
  ROUND(AVG(quality_score), 2) AS avg_quality_score
FROM consulting_delivery_performance
GROUP BY
  employee_id,
  department,
  job_level
ORDER BY employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	utilization_pct	revenue_per_billable_hour	contribution_proxy	rework_rate_pct	avg_quality_score
E0001	Analytics	L4	73.19	1818.62	455802.94	1.58	86
E0001	Operations	L3	88.01	2597.29	478804.35	13.86	92
E0001	Strategy	L1	69.91	2315.57	727938.26	4.17	83
E0001	Strategy	L5	80.16	1164.97	90802.02	3.41	84
E0001	Technology	L3	70.55	1872.54	620179.41	4.51	70.5
E0002	Analytics	L5	86.45	1702.26	389757.19	6.23	72
E0002	Technology	L4	41.15	2383.42	107546.65	12.5	86
E0003	Analytics	L3	8	2432.23	-75115.77	8	79
E0003	Analytics	L6	34.24	1960.56	-40462.61	4.67	83
E0003	Strategy	L2	38.24	1049.82	-93797.49	11.18	90
E0003	Technology	L1	77.89	2302.33	324706.58	10.88	72
E0003	Technology	L3	29.53	1569.02	-28334.71	7.8	82.5
E0004	Analytics	L1	51.04	1637.48	19969.36	15.63	76
E0004	Strategy	L6	73.1	2270.09	546831.08	3.54	76
E0004	Technology	L3	94.08	2101.48	459195.93	5.33	100
E0004	Technology	L5	85	1073.95	160202.32	12.35	72
E0005	Analytics	L3	67.98	1709.37	229454.24	22.19	84.5

Step 2 — Peer-group ranking

Revenue per billable hour and quality are ranked with RANK() inside each department × job level × client tier × project complexity partition. Most excerpt rows are rank 1 because many peer groups contain a single employee.

QUESTION 1 · STEP 2 — PEER-GROUP RANKING

```sql
WITH employee_metrics AS (
  SELECT
      employee_id,
      department,
      job_level,
      client_tier,
      project_complexity,
      SUM(recognized_revenue) /
         NULLIF(SUM(billable_hours), 0)
         AS revenue_per_billable_hour,
      AVG(quality_score) AS avg_quality_score
    FROM consulting_delivery_performance
    GROUP BY
      employee_id,
      department,
      job_level,
      client_tier,
      project_complexity
)
SELECT
    employee_id,
    department,
    job_level,
    client_tier,
    project_complexity,
    ROUND(revenue_per_billable_hour, 2)
      AS revenue_per_billable_hour,
    ROUND(avg_quality_score, 2)
      AS avg_quality_score,
    RANK() OVER (
      PARTITION BY
         department,
         job_level,
      client_tier,
      project_complexity
    ORDER BY revenue_per_billable_hour DESC
  ) AS revenue_peer_rank,
  RANK() OVER (
    PARTITION BY
      department,
      job_level,
      client_tier,
      project_complexity
    ORDER BY avg_quality_score DESC
```

```
  ) AS quality_peer_rank
FROM employee_metrics
ORDER BY
  department,
  job_level,
  revenue_peer_rank;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	client_tier	project_complexity	revenue_per_billable_hour	avg_quality_score	revenue_peer_rank	quality_peer_rank
E0046	Analytics	L1	Enterprise	1	1902.28	89	1	1
E0069	Analytics	L1	Enterprise	3	2226.66	94	1	1
E0033	Analytics	L1	Enterprise	4	1813.39	73	1	2
E0042	Analytics	L1	Enterprise	6	1593.25	76	1	1
E0118	Analytics	L1	Enterprise	7	2024.24	87	1	1
E0054	Analytics	L1	Enterprise	8	2597.99	75	1	2
E0118	Analytics	L1	Enterprise	9	2382.4	94	1	2
E0089	Analytics	L1	Mid-Market	3	2511.31	77	1	1
E0004	Analytics	L1	Mid-Market	4	1637.48	76	1	2
E0065	Analytics	L1	Mid-Market	5	2129.18	77	1	2
E0111	Analytics	L1	Mid-Market	6	2134.85	68	1	1
E0119	Analytics	L1	Mid-Market	7	1548.47	77	1	1
E0045	Analytics	L1	Mid-Market	8	1061.26	77	1	2
E0109	Analytics	L1	Strategic	3	2596.1	81	1	1
E0010	Analytics	L1	Strategic	7	1625.16	93	1	1

Step 3 — Top 10% revenue with below-median quality

NTILE(10) marks the top revenue decile; a ROW_NUMBER() median of employee quality (82.5) is cross-joined to keep only decile-one employees below it. Rows run from 2,597.99 down to 2,482.28 revenue per billable hour, with quality between 62 and 81.

QUESTION 1 · STEP 3 — TOP 10% REVENUE WITH BELOW-MEDIAN QUALITY

```sql
WITH employee_metrics AS (
  SELECT
    employee_id,
    department,
       job_level,
       SUM(recognized_revenue) /
            NULLIF(SUM(billable_hours), 0)
            AS revenue_per_billable_hour,
       AVG(quality_score) AS avg_quality_score
     FROM consulting_delivery_performance
     GROUP BY
       employee_id,
       department,
       job_level
),
ranked_employees AS (
     SELECT
       *,
       NTILE(10) OVER (
            ORDER BY revenue_per_billable_hour DESC
       ) AS revenue_decile
     FROM employee_metrics
),
quality_median AS (
     SELECT AVG(avg_quality_score) AS median_quality
     FROM (
       SELECT
            avg_quality_score,
            ROW_NUMBER() OVER (
              ORDER BY avg_quality_score
            ) AS rn,
            COUNT(*) OVER () AS total_count
       FROM employee_metrics
     )q
     WHERE rn IN (
         FLOOR((total_count + 1) / 2),
         FLOOR((total_count + 2) / 2)
     )
)
SELECT
     r.employee_id,
     r.department,
     r.job_level,
     ROUND(r.revenue_per_billable_hour, 2)

         AS revenue_per_billable_hour,
     ROUND(r.avg_quality_score, 2)
         AS avg_quality_score,
     ROUND(m.median_quality, 2) AS median_quality
FROM ranked_employees r
CROSS JOIN quality_median m
WHERE r.revenue_decile = 1
    AND r.avg_quality_score < m.median_quality
ORDER BY r.revenue_per_billable_hour DESC;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	revenue_per_billable_hour	avg_quality_score	median_quality
E0054	Analytics	L1	2597.99	75	82.5
E0109	Analytics	L1	2596.1	81	82.5
E0070	Operations	L6	2594.55	66	82.5
E0109	Technology	L5	2593.51	69	82.5
E0112	Operations	L3	2588.49	71	82.5
E0063	Technology	L2	2581.23	75	82.5
E0078	Operations	L1	2566.14	65	82.5
E0017	Operations	L1	2542.99	70	82.5
E0040	Operations	L4	2523.42	70	82.5
E0045	Strategy	L3	2519.63	77	82.5
E0089	Analytics	L1	2511.31	77	82.5
E0030	Strategy	L2	2508.56	72	82.5
E0036	Operations	L2	2499.89	81	82.5
E0119	Strategy	L3	2486.47	62	82.5
E0038	Technology	L6	2482.28	73	82.5

Step 4 — Company-wide rank versus peer-group rank

Both ranks side by side. E0001 in Operations (L3) is second company-wide at 2,597.29 and first among peers; E0001 in Technology (L3) at 901.01 is 629th company-wide but only second in its peer group.

QUESTION 1 · STEP 4 — COMPANY-WIDE RANK VERSUS PEER-GROUP RANK

```
WITH employee_metrics AS (
     SELECT
      employee_id,
      department,
      job_level,
      client_tier,
      project_complexity,
      SUM(recognized_revenue) /
           NULLIF(SUM(billable_hours), 0)
           AS revenue_per_billable_hour,
      SUM(billable_hours) * 100.0 /
           NULLIF(
             SUM(billable_hours + non_billable_hours), 0
           ) AS utilization_pct,
      AVG(quality_score) AS quality_score
     FROM consulting_delivery_performance
     GROUP BY
      employee_id,
      department,
      job_level,
      client_tier,
      project_complexity
),
ranked_employees AS (
     SELECT
      *,
      RANK() OVER (
           ORDER BY revenue_per_billable_hour DESC
      ) AS company_rank,
      RANK() OVER (
        PARTITION BY
          department,
          job_level,
          client_tier,
          project_complexity
        ORDER BY revenue_per_billable_hour DESC
      ) AS peer_rank
    FROM employee_metrics
)
SELECT
    employee_id,
    department,
    job_level,
    ROUND(revenue_per_billable_hour, 2)
```

```
      AS revenue_per_billable_hour,
    ROUND(utilization_pct, 2) AS utilization_pct,
    ROUND(quality_score, 2) AS quality_score,
    company_rank,
    peer_rank
FROM ranked_employees
ORDER BY employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	revenue_per_billable_hour	utilization_pct	quality_score	company_rank	peer_rank
E0001	Analytics	L4	1818.62	73.19	86	304	2
E0001	Technology	L3	2233.75	83.72	64	147	2
E0001	Technology	L3	901.01	49.57	77	629	2
E0001	Operations	L3	2597.29	88.01	92	2	1
E0001	Strategy	L5	1164.97	80.16	84	534	2
E0001	Strategy	L1	2315.57	69.91	83	112	1
E0002	Technology	L4	2383.42	41.15	86	82	1
E0002	Analytics	L5	1702.26	86.45	72	342	1
E0003	Analytics	L6	1960.56	34.24	83	250	1
E0003	Analytics	L3	2432.23	8	79	70	1
E0003	Technology	L3	1779.51	37.11	81	322	1
E0003	Technology	L1	2302.33	77.89	72	117	2
E0003	Technology	L3	1304.8	23.5	84	488	3
E0003	Strategy	L2	1049.82	38.24	90	579	2
E0004	Strategy	L6	2270.09	73.1	76	132	1

KEY INSIGHT

Peer-group ranking changes the story most for mid-table consultants: a record that is 629th company-wide can still be second in its own group, and a top-two company rank (E0001, Operations L3, 2,597.29) is first in its group. Step 3 isolates the highest earners per billable hour that sit below the 82.5 median quality score, for example E0054 (Analytics L1, 2,597.99 with quality 75) and E0070 (Operations L6, 2,594.55 with quality 66).

04
SECTION

Performance Trajectory

Question 2: quality declines, running department benchmarks and the check for productivity gains that are really a change of project mix.

Question 2 — Performance trajectory and sustained underperformance

Identify employees whose quality score declines across at least three consecutive project records; employees whose revenue per billable hour rises while quality-adjusted productivity falls; compare each employee with the department benchmark available at the time; and flag improvement that coincides with a change in complexity or client tier.

Step 1 — Consecutive quality declines

LAG() marks each decline; a running count of non-decline rows labels each run, and the longest run is kept per employee. E0010 and E0120 reach four consecutive declines and ten more employees in the excerpt reach three.

QUESTION 2 · STEP 1 — CONSECUTIVE QUALITY DECLINES

```
WITH project_comparison AS (
     SELECT
      employee_id,
      record_id,
      project_start_date,
      quality_score,
      LAG(quality_score) OVER (
           PARTITION BY employee_id
           ORDER BY project_start_date, record_id
      ) AS previous_quality_score
     FROM consulting_delivery_performance
),
quality_changes AS (
     SELECT
      *,
      CASE
           WHEN quality_score < previous_quality_score THEN 1
           ELSE 0
      END AS is_decline
     FROM project_comparison
),
decline_streaks AS (
     SELECT
       *,
       SUM(
            CASE
              WHEN is_decline = 0 THEN 1
              ELSE 0
            END
       ) OVER (
            PARTITION BY employee_id
            ORDER BY project_start_date, record_id
       ) AS streak_group
     FROM quality_changes
),
streak_counts AS (
     SELECT
       employee_id,
       streak_group,
       COUNT(*) AS consecutive_quality_declines
     FROM decline_streaks
     WHERE is_decline = 1
     GROUP BY employee_id, streak_group
)
```

```sql
SELECT
     employee_id,
     MAX(consecutive_quality_declines)
       AS consecutive_quality_declines
FROM streak_counts
GROUP BY employee_id
HAVING MAX(consecutive_quality_declines) >= 3
ORDER BY consecutive_quality_declines DESC;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	consecutive_quality_declines
E0010	4
E0120	4
E0011	3
E0012	3
E0014	3
E0021	3
E0038	3
E0056	3
E0074	3
E0090	3
E0093	3
E0109	3

Step 2 — Project count and latest quality

ROW_NUMBER() in descending date order picks each employee's latest project; COUNT(*) OVER () gives the number of projects.

QUESTION 2 · STEP 2 — PROJECT COUNT AND LATEST QUALITY

```sql
WITH ranked_projects AS (
    SELECT
      employee_id,
      record_id,
      project_start_date,
      quality_score,
      ROW_NUMBER() OVER (
        PARTITION BY employee_id
        ORDER BY project_start_date DESC, record_id DESC
      ) AS rn,
      COUNT(*) OVER (
        PARTITION BY employee_id
      ) AS project_count
    FROM consulting_delivery_performance
)
SELECT
    employee_id,
    MAX(project_count) AS project_count,
    MAX(CASE WHEN rn = 1 THEN quality_score END) AS latest_quality
FROM ranked_projects
GROUP BY employee_id
ORDER BY employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	project_count	latest_quality
E0001	6	83
E0002	2	86
E0003	6	90
E0004	4	76
E0005	7	76
E0006	7	93
E0007	7	88
E0008	6	84
E0009	7	86
E0010	5	84
E0011	7	100
E0012	5	86
E0013	3	60
E0014	5	63
E0015	6	80

Step 3 — Historical department average (running)

A running average of quality within each department, ordered by start date, gives the benchmark that was available at each point in time.

QUESTION 2 · STEP 3 — HISTORICAL DEPARTMENT AVERAGE (RUNNING)

```sql
SELECT
  employee_id,
  record_id,
  department,
  project_start_date,
  quality_score,
  ROUND(
    AVG(quality_score) OVER (
         PARTITION BY department
         ORDER BY project_start_date, record_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ),
    2
  ) AS historical_department_avg
FROM consulting_delivery_performance
ORDER BY employee_id, project_start_date, record_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	record_id	department	project_start_date	quality_score	historical_department_avg
E0001	R00691	Technology	04-05-2023	77	82.06
E0001	R00107	Analytics	05-06-2024	86	83.29
E0001	R00442	Strategy	08-06-2024	84	82.16
E0001	R00134	Operations	16-11-2024	92	81.5
E0001	R00644	Technology	30-08-2025	64	82.59
E0001	R00303	Strategy	15-11-2025	83	82.16
E0002	R00330	Analytics	29-09-2024	72	82.86
E0002	R00036	Technology	19-02-2025	86	82.92
E0003	R00575	Analytics	07-02-2024	79	83.29
E0003	R00146	Technology	02-03-2024	84	83.95
E0003	R00351	Technology	02-04-2025	72	82.58
E0003	R00471	Technology	02-05-2025	81	82.43
E0003	R00055	Analytics	12-10-2025	83	82.41
E0003	R00217	Strategy	17-10-2025	90	82.07
E0004	R00527	Strategy	25-01-2023	76	79

Step 4 — Relative productivity against the running benchmark

Revenue per billable hour minus the running department average, so a project is only compared with what the department had achieved before it.

QUESTION 2 · STEP 4 — RELATIVE PRODUCTIVITY AGAINST THE RUNNING BENCH MARK

```sql
WITH project_productivity AS (
     SELECT
      employee_id,
      record_id,
      department,
      project_start_date,
      billable_hours,
      recognized_revenue,
      quality_score,
      recognized_revenue / NULLIF(billable_hours, 0)
        AS revenue_per_billable_hour
     FROM consulting_delivery_performance
),
department_benchmark AS (
     SELECT
      *,
      AVG(revenue_per_billable_hour) OVER (
           PARTITION BY department
           ORDER BY project_start_date, record_id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
      ) AS historical_department_revenue_avg
    FROM project_productivity
)
SELECT
    employee_id,
    record_id,
    department,
    project_start_date,
    ROUND(revenue_per_billable_hour, 2)
      AS revenue_per_billable_hour,
    ROUND(historical_department_revenue_avg, 2)
      AS historical_department_revenue_avg,
    ROUND(
      revenue_per_billable_hour
      - historical_department_revenue_avg,
      2
    ) AS relative_productivity
FROM department_benchmark
ORDER BY employee_id, project_start_date, record_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	record_id	department	project_start_date	revenue_per_billable_hour	historical_department_revenue_avg	relative_productivity
E0001	R00691	Technology	04-05-2023	901.01	1631.78	-730.76
E0001	R00107	Analytics	05-06-2024	1818.62	1637.86	180.76
E0001	R00442	Strategy	08-06-2024	1164.97	1684.48	-519.51
E0001	R00134	Operations	16-11-2024	2597.29	1643.66	953.63
E0001	R00644	Technology	30-08-2025	2233.75	1788.67	445.08
E0001	R00303	Strategy	15-11-2025	2315.57	1690.76	624.81
E0002	R00330	Analytics	29-09-2024	1702.26	1627.33	74.93
E0002	R00036	Technology	19-02-2025	2383.42	1769.64	613.78
E0003	R00575	Analytics	07-02-2024	2432.23	1642.08	790.14
E0003	R00146	Technology	02-03-2024	1304.8	1658.74	-353.94
E0003	R00351	Technology	02-04-2025	2302.33	1769.97	532.37
E0003	R00471	Technology	02-05-2025	1779.51	1783.71	-4.2
E0003	R00055	Analytics	12-10-2025	1960.56	1639.26	321.3
E0003	R00217	Strategy	17-10-2025	1049.82	1688.09	-638.28
E0004	R00527	Strategy	25-01-2023	2270.09	1549.47	720.63

Step 5 — Trajectory summary

Combines the decline streak, latest quality, running department quality average and latest relative productivity per employee. In the excerpt E0010 shows four declines with latest quality 84, E0011 and E0012 show three, and E0011 closes at 100 quality with relative productivity of −840.72.

QUESTION 2 · STEP 5 — TRAJECTORY SUMMARY

```
WITH project_data AS (
  SELECT
    employee_id,
    record_id,
    project_start_date,
    department,
    billable_hours,
    recognized_revenue,
    quality_score,
    LAG(quality_score) OVER (
      PARTITION BY employee_id
      ORDER BY project_start_date, record_id
    ) AS previous_quality,
    ROW_NUMBER() OVER (
      PARTITION BY employee_id
      ORDER BY project_start_date DESC, record_id DESC
    ) AS latest_row,
    COUNT(*) OVER (
      PARTITION BY employee_id
    ) AS project_count,
      AVG(quality_score) OVER (
           PARTITION BY department
           ORDER BY project_start_date, record_id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
      ) AS historical_department_avg,
      AVG(
           recognized_revenue / NULLIF(billable_hours, 0)
      ) OVER (
           PARTITION BY department
           ORDER BY project_start_date, record_id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
      ) AS department_revenue_avg
     FROM consulting_delivery_performance
),
decline_flags AS (
     SELECT
      *,
      CASE
           WHEN quality_score < previous_quality THEN 1
           ELSE 0
      END AS is_decline
     FROM project_data
),
streak_groups AS (
```

```
     SELECT
      *,
      SUM(
           CASE WHEN is_decline = 0 THEN 1 ELSE 0 END
      ) OVER (
           PARTITION BY employee_id
           ORDER BY project_start_date, record_id
       ) AS streak_group
     FROM decline_flags
),
streak_counts AS (
     SELECT
       employee_id,
       streak_group,
       COUNT(*) AS decline_count
     FROM streak_groups
     WHERE is_decline = 1
     GROUP BY employee_id, streak_group
),
max_streak AS (
     SELECT
       employee_id,
       MAX(decline_count) AS consecutive_quality_declines
     FROM streak_counts
     GROUP BY employee_id
),
employee_summary AS (
     SELECT
       employee_id,
       MAX(project_count) AS project_count,
       MAX(
         CASE WHEN latest_row = 1 THEN quality_score END
       ) AS latest_quality,
       MAX(
         CASE WHEN latest_row = 1
              THEN historical_department_avg END
       ) AS historical_department_avg,
       MAX(
        CASE WHEN latest_row = 1 THEN
           recognized_revenue / NULLIF(billable_hours, 0)
           - department_revenue_avg
        END
      ) AS relative_productivity
    FROM project_data

    GROUP BY employee_id
)
SELECT
    e.employee_id,
    e.project_count,
    COALESCE(m.consecutive_quality_declines, 0)
      AS consecutive_quality_declines,
    e.latest_quality,
    ROUND(e.historical_department_avg, 2)
      AS historical_department_avg,
    ROUND(e.relative_productivity, 2)
      AS relative_productivity
FROM employee_summary e
LEFT JOIN max_streak m
    ON e.employee_id = m.employee_id
ORDER BY e.employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	project_count	consecutive_quality_declines	latest_quality	historical_department_avg	relative_productivity
E0001	6	1	83	82.16	624.81
E0002	2	0	86	82.92	613.78
E0003	6	1	90	82.07	-638.28
E0004	4	1	76	82.83	22.29
E0005	7	1	76	81.23	391.17
E0006	7	1	93	81.24	86.35
E0007	7	2	88	81.34	-839.94
E0008	6	1	84	81.54	54.42
E0009	7	1	86	82.64	226.84
E0010	5	4	84	83.26	255.45
E0011	7	3	100	82.83	-840.72
E0012	5	3	86	82.47	578.23
E0013	3	1	60	83.17	-790.46
E0014	5	3	63	82.46	691.67
E0015	6	2	80	82.71	199.58

Step 6 — Consecutive project comparison

Flags a project when revenue per hour rose but revenue per hour × quality fell, and separately when complexity or client tier differs from the previous project. None of the excerpt rows trip the productivity flag; most later projects carry the complexity/tier-change flag.

QUESTION 2 · STEP 6 — CONSECUTIVE PROJECT COMPARISON

```sql
WITH project_comparison AS (
  SELECT
    employee_id,
    record_id,
    project_start_date,
    project_complexity,
    client_tier,
    recognized_revenue,
    billable_hours,
    quality_score,
    recognized_revenue / NULLIF(billable_hours, 0)
      AS revenue_per_billable_hour,
    LAG(
      recognized_revenue / NULLIF(billable_hours, 0)
    ) OVER (
      PARTITION BY employee_id
      ORDER BY project_start_date, record_id
    ) AS previous_revenue_per_hour,
    LAG(quality_score) OVER (
      PARTITION BY employee_id
      ORDER BY project_start_date, record_id
    ) AS previous_quality,
    LAG(project_complexity) OVER (
      PARTITION BY employee_id
      ORDER BY project_start_date, record_id
    ) AS previous_complexity,
    LAG(client_tier) OVER (
        PARTITION BY employee_id
        ORDER BY project_start_date, record_id
      ) AS previous_client_tier
    FROM consulting_delivery_performance
)
SELECT
    employee_id,
    record_id,
    ROUND(previous_revenue_per_hour, 2) AS previous_revenue_per_hour,
    ROUND(revenue_per_billable_hour, 2) AS current_revenue_per_hour,
    previous_quality,
    quality_score AS current_quality,
    CASE
      WHEN revenue_per_billable_hour > previous_revenue_per_hour
      AND revenue_per_billable_hour * quality_score
         < previous_revenue_per_hour * previous_quality
      THEN 'Yes'
```

```
      ELSE 'No'
    END AS revenue_up_quality_adjusted_productivity_down,
    CASE
      WHEN project_complexity <> previous_complexity
       OR client_tier <> previous_client_tier
      THEN 'Check complexity/client tier change'
      ELSE 'No change'
    END AS complexity_tier_flag
FROM project_comparison
ORDER BY employee_id, project_start_date, record_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	record_id	previous_revenue_per_hour	current_revenue_per_hour	previous_quality	current_quality	revenue_up_quality_adjusted_productivity_down	complexity_tier_flag
E0001	R00691	NULL	901.01	NULL	77	No	No change
E0001	R00107	901.01	1818.62	77	86	No	Check complexity/client tier change
E0001	R00442	1818.62	1164.97	86	84	No	Check complexity/client tier change
E0001	R00134	1164.97	2597.29	84	92	No	No change
E0001	R00644	2597.29	2233.75	92	64	No	Check complexity/client tier change
E0001	R00303	2233.75	2315.57	64	83	No	Check complexity/client tier change
E0002	R00330	NULL	1702.26	NULL	72	No	No change
E0002	R00036	1702.26	2383.42	72	86	No	Check complexity/client tier change
E0003	R00575	NULL	2432.23	NULL	79	No	No change
E0003	R00146	2432.23	1304.8	79	84	No	Check complexity/client tier change
E0003	R00351	1304.8	2302.33	84	72	No	Check complexity/client tier change
E0003	R00471	2302.33	1779.51	72	81	No	No change
E0003	R00055	1779.51	1960.56	81	83	No	Check complexity/client tier change
E0003	R00217	1960.56	1049.82	83	90	No	Check complexity/client tier change
E0004	R00527	NULL	2270.09	NULL	76	No	No change
E0004	R00329	2270.09	1073.95	76	72	No	Check complexity/client tier change
E0004	R00279	1073.95	2101.48	72	100	No	Check complexity/client tier change

KEY INSIGHT

Sustained decline is rare and concentrated: only two employees reach four consecutive quality declines and the next group stops at three. The complexity/tier flag fires on most projects after an employee's first, which is why apparent improvement should be read against project mix before it is rewarded.

05
SECTION

Counterfactual Staffing

Question 3: which segments to take hours from, which to give them to, and how much of the estimated gain survives a change of population.

Question 3 — Counterfactual staffing and resource allocation

Identify segments with high delivery cost, elevated rework and below-median quality. Estimate the contribution change if 10% of delivery hours move from bottom-performing segments to segments with above-median contribution per billable hour, rank departments, and run sensitivity checks excluding exited employees and projects under 100 billable hours.

Step 1 — Segment-level performance

Twelve department × client-tier segments, ordered by contribution per billable hour. The spread is 559.02 to 933.44, a factor of 1.67.

QUESTION 3 · STEP 1 — SEGMENT-LEVEL PERFORMANCE

```sql
SELECT
  department,
  client_tier,
  COUNT(*) AS total_records,
  ROUND(SUM(recognized_revenue), 2)
    AS total_revenue,
  ROUND(SUM(delivery_cost), 2)
    AS total_delivery_cost,
  ROUND(SUM(recognized_revenue - delivery_cost), 2)
    AS contribution_proxy,
  ROUND(
    SUM(recognized_revenue - delivery_cost)
    / NULLIF(SUM(billable_hours), 0),
    2
  ) AS contribution_per_billable_hour,
  ROUND(AVG(rework_hours), 2)
    AS avg_rework_hours,
  ROUND(AVG(quality_score), 2)
    AS avg_quality_score
FROM consulting_delivery_performance
GROUP BY department, client_tier
ORDER BY contribution_per_billable_hour;
```

OUTPUT — AS EXECUTED

department	client_tier	records	total_revenue	delivery_cost	contribution	contrib_per_hr	avg_rework	avg_quality
Strategy	Mid-Market	51	17,345,363.43	10,760,113.34	6,585,250.09	559.02	33.33	81.71
Operations	Mid-Market	60	23,558,199.89	14,685,667.55	8,872,532.34	614.53	32.20	81.02
Analytics	Enterprise	98	35,812,054.18	21,309,789.01	14,502,265.17	650.24	32.66	82.96
Operations	Strategic	28	11,702,585.99	6,983,909.17	4,718,676.82	677.39	31.14	80.96
Analytics	Mid-Market	63	24,028,038.38	14,163,720.67	9,864,317.71	679.41	36.24	81.94
Technology	Strategic	27	12,110,604.03	6,632,054.32	5,478,549.71	721.05	29.48	83.44
Analytics	Strategic	35	12,542,604.82	7,024,437.37	5,518,167.45	721.52	34.89	81.86
Strategy	Enterprise	82	34,863,158.08	19,698,408.16	15,164,749.92	736.76	34.93	82.79
Strategy	Strategic	27	11,127,051.67	6,328,736.51	4,798,315.16	757.67	28.33	81.07
Operations	Enterprise	88	37,798,884.50	20,323,480.69	17,475,403.81	761.75	36.85	81.53
Technology	Mid-Market	55	22,601,758.24	12,088,198.82	10,513,559.42	820.54	35.85	82.60
Technology	Enterprise	86	39,414,565.46	18,830,387.49	20,584,177.97	933.44	34.33	82.13

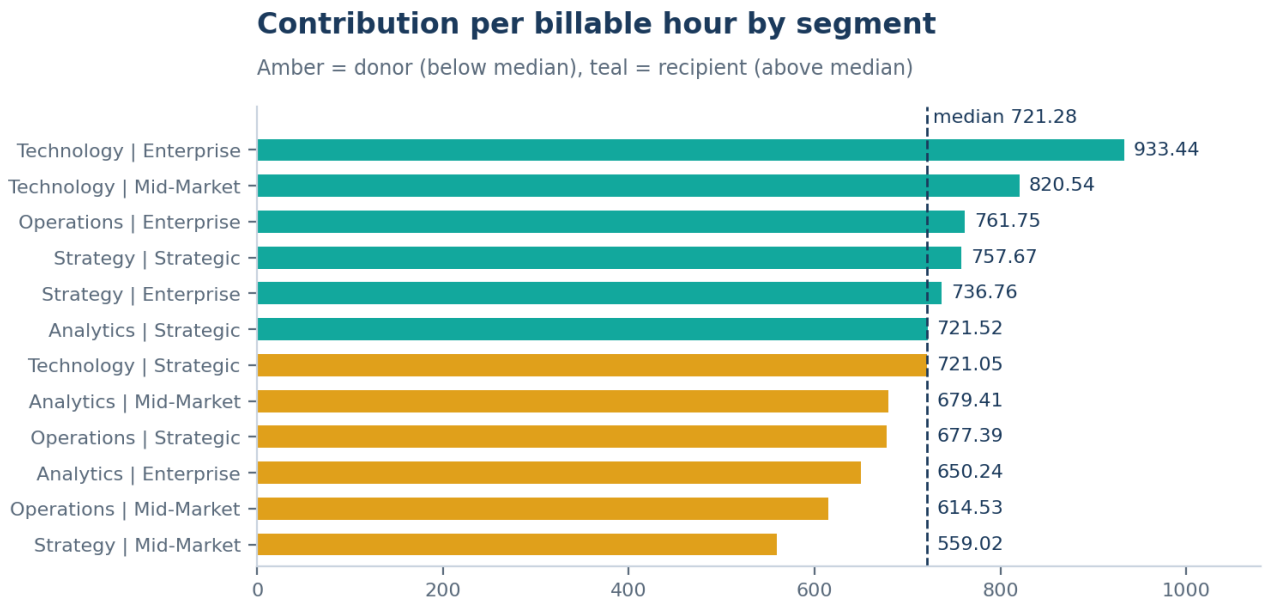


Figure 5.1 — Contribution per billable hour by segment against the 721.28 median (Question 3, Step 1).

Step 2 — Under-performing segments

Segments ordered by lowest quality, then highest rework, then highest delivery cost. Operations × Strategic is the weakest on quality (80.96) with the highest average delivery cost (249,425.33); Operations × Mid-Market follows at 81.02.

QUESTION 3 · STEP 2 — UNDER-PERFORMING SEGMENTS

```sql
SELECT
  department,
  client_tier,
  ROUND(AVG(delivery_cost), 2) AS avg_delivery_cost,
  ROUND(AVG(rework_hours), 2) AS avg_rework_hours,
  ROUND(AVG(quality_score), 2) AS avg_quality_score,
  ROUND(
    SUM(recognized_revenue - delivery_cost)
    / NULLIF(SUM(billable_hours), 0),
    2
  ) AS contribution_per_billable_hour
FROM consulting_delivery_performance
GROUP BY department, client_tier
ORDER BY
  avg_quality_score ASC,
     avg_rework_hours DESC,
     avg_delivery_cost DESC;
```

OUTPUT — AS EXECUTED

department	client_tier	avg_delivery_cost	avg_rework_hours	avg_quality_score	contribution_per_billable_hour
Operations	Strategic	249,425.33	31.14	80.96	677.39
Operations	Mid-Market	244,761.13	32.20	81.02	614.53
Strategy	Strategic	234,397.65	28.33	81.07	757.67
Operations	Enterprise	230,948.64	36.85	81.53	761.75
Strategy	Mid-Market	210,982.61	33.33	81.71	559.02
Analytics	Strategic	200,698.21	34.89	81.86	721.52
Analytics	Mid-Market	224,820.96	36.24	81.94	679.41
Technology	Enterprise	218,957.99	34.33	82.13	933.44
Technology	Mid-Market	219,785.43	35.85	82.60	820.54
Strategy	Enterprise	240,224.49	34.93	82.79	736.76
Analytics	Enterprise	217,446.83	32.66	82.96	650.24
Technology	Strategic	245,631.64	29.48	83.44	721.05

Step 3 — Median contribution per billable hour

The median of the twelve segment rates is the average of the 6th and 7th values (721.05 and 721.52).

QUESTION 3 · STEP 3 — MEDIAN CONTRIBUTION PER BILLABLE HOUR

```
WITH segment_performance AS (
     SELECT
       department,
       client_tier,
       SUM(recognized_revenue - delivery_cost)
         / NULLIF(SUM(billable_hours), 0)
         AS contribution_per_billable_hour
     FROM consulting_delivery_performance
     GROUP BY department, client_tier
),
ranked_segments AS (
     SELECT
       contribution_per_billable_hour,
       ROW_NUMBER() OVER (
         ORDER BY contribution_per_billable_hour
       ) AS rn,
       COUNT(*) OVER () AS total_segments
     FROM segment_performance
     WHERE contribution_per_billable_hour IS NOT NULL
)
SELECT
     ROUND(AVG(contribution_per_billable_hour), 2)
       AS median_contribution_per_billable_hour
FROM ranked_segments
WHERE rn IN (
     FLOOR((total_segments + 1) / 2),
     FLOOR((total_segments + 2) / 2)
);
```

OUTPUT — AS EXECUTED

median_contribution_per_billable_hour
721.28

Step 4 — Contribution after a 10% reallocation

Segments below the median are donors and give up 10% of their billable hours; the pooled hours go to above-median recipients in proportion to their own hours, earning each recipient's current contribution per hour.

QUESTION 3 · STEP 4 — CONTRIBUTION AFTER A 10% REALLOCATION

```
WITH segment_performance AS (
     SELECT
       department,
       client_tier,
       SUM(billable_hours) AS billable_hours,
       SUM(recognized_revenue - delivery_cost) AS contribution,
       SUM(recognized_revenue - delivery_cost)
            / NULLIF(SUM(billable_hours), 0) AS contribution_per_hour
     FROM consulting_delivery_performance
     GROUP BY department, client_tier
),
ranked_segments AS (
     SELECT
       *,
       ROW_NUMBER() OVER (
            ORDER BY contribution_per_hour
         ) AS rn,
         COUNT(*) OVER () AS total_segments
     FROM segment_performance
     WHERE contribution_per_hour IS NOT NULL
),
median_value AS (
     SELECT AVG(contribution_per_hour) AS median_rate
     FROM ranked_segments
     WHERE rn IN (
         FLOOR((total_segments + 1) / 2),
         FLOOR((total_segments + 2) / 2)
     )
),
classified AS (
     SELECT
         s.*,
         CASE
           WHEN s.contribution_per_hour < m.median_rate
                THEN 'Donor'
           WHEN s.contribution_per_hour > m.median_rate
                THEN 'Recipient'
           ELSE 'Median'
         END AS segment_group
     FROM segment_performance s
     CROSS JOIN median_value m
),
pool AS (
     SELECT
```

```
          SUM(
            CASE WHEN segment_group = 'Donor'
            THEN billable_hours * 0.10 ELSE 0 END
        ) AS donated_hours
      FROM classified
),
recipients AS (
      SELECT SUM(billable_hours) AS recipient_hours
      FROM classified
      WHERE segment_group = 'Recipient'
)
SELECT
      c.department,
      c.client_tier,
      ROUND(c.contribution, 2) AS baseline_contribution,
      ROUND(
        c.contribution
        - CASE
            WHEN c.segment_group = 'Donor'
            THEN c.billable_hours * 0.10 * c.contribution_per_hour
            ELSE 0
          END
        + CASE
            WHEN c.segment_group = 'Recipient'
            THEN p.donated_hours
              * c.billable_hours
              / NULLIF(r.recipient_hours, 0)
              * c.contribution_per_hour
            ELSE 0
          END,
        2
      ) AS contribution_after_reallocation
FROM classified c
CROSS JOIN pool p
CROSS JOIN recipients r
ORDER BY c.department, c.client_tier;
```

OUTPUT — AS EXECUTED

department	client_tier	baseline_contribution	contribution_after_reallocation	role
Analytics	Enterprise	14,502,265.17	13,052,038.65	Donor
Analytics	Mid-Market	9,864,317.71	8,877,885.94	Donor
Analytics	Strategic	5,518,167.45	5,981,772.37	Recipient
Operations	Enterprise	17,475,403.81	18,943,587.46	Recipient
Operations	Mid-Market	8,872,532.34	7,985,279.11	Donor
Operations	Strategic	4,718,676.82	4,246,809.14	Donor
Strategy	Enterprise	15,164,749.92	16,438,805.62	Recipient
Strategy	Mid-Market	6,585,250.09	5,926,725.08	Donor
Strategy	Strategic	4,798,315.16	5,201,442.20	Recipient
Technology	Enterprise	20,584,177.97	22,313,543.07	Recipient
Technology	Mid-Market	10,513,559.42	11,396,848.65	Recipient
Technology	Strategic	5,478,549.71	4,930,694.74	Donor

Step 5 — Department-level scenario

Segment results rolled up to departments and ranked by contribution change.

QUESTION 3 · STEP 5 — DEPARTMENT-LEVEL SCENARIO

```
WITH segment_performance AS (
     SELECT
       department,
       client_tier,
       SUM(billable_hours) AS billable_hours,
       SUM(recognized_revenue - delivery_cost) AS contribution,
       SUM(recognized_revenue - delivery_cost)
            / NULLIF(SUM(billable_hours), 0) AS contribution_per_hour
     FROM consulting_delivery_performance
     GROUP BY department, client_tier
),
ranked_segments AS (
     SELECT
       *,
       ROW_NUMBER() OVER (
            ORDER BY contribution_per_hour
       ) AS rn,
         COUNT(*) OVER () AS total_segments
     FROM segment_performance
     WHERE contribution_per_hour IS NOT NULL
),
median_value AS (
     SELECT AVG(contribution_per_hour) AS median_rate
     FROM ranked_segments
     WHERE rn IN (
         FLOOR((total_segments + 1) / 2),
         FLOOR((total_segments + 2) / 2)
     )
),
classified AS (
     SELECT
         s.*,
         CASE
           WHEN s.contribution_per_hour < m.median_rate
                THEN 'Donor'
           WHEN s.contribution_per_hour > m.median_rate
                THEN 'Recipient'
           ELSE 'Median'
         END AS segment_group
     FROM segment_performance s
     CROSS JOIN median_value m
),
pool AS (
     SELECT
```

```
          SUM(
            CASE WHEN segment_group = 'Donor'
            THEN billable_hours * 0.10 ELSE 0 END
          ) AS donated_hours
      FROM classified
),
recipients AS (
      SELECT SUM(billable_hours) AS recipient_hours
      FROM classified
      WHERE segment_group = 'Recipient'
),
scenario AS (
      SELECT
        c.department,
        c.contribution AS baseline_contribution,
        c.contribution
        - CASE
          WHEN c.segment_group = 'Donor'
          THEN c.billable_hours * 0.10 * c.contribution_per_hour
          ELSE 0
         END
        + CASE
          WHEN c.segment_group = 'Recipient'
          THEN p.donated_hours
               * c.billable_hours
               / NULLIF(r.recipient_hours, 0)
               * c.contribution_per_hour
          ELSE 0
         END AS contribution_after_reallocation
      FROM classified c
      CROSS JOIN pool p
      CROSS JOIN recipients r
)
SELECT
      department,
  ROUND(SUM(baseline_contribution), 2)
    AS baseline_contribution,
  ROUND(SUM(contribution_after_reallocation), 2)
    AS contribution_after_reallocation,
  ROUND(
    SUM(contribution_after_reallocation)
    - SUM(baseline_contribution), 2
  ) AS contribution_change,
  ROUND(

    (
        SUM(contribution_after_reallocation)
        - SUM(baseline_contribution)
    ) / NULLIF(SUM(baseline_contribution), 0) * 100,
    2
  ) AS change_pct
FROM scenario
GROUP BY department
ORDER BY contribution_change DESC;
```

OUTPUT — AS EXECUTED

department	baseline_contribution	contribution_after_reallocation	contribution_change	change_pct
Technology	36,576,287.10	38,641,086.47	2,064,799.37	5.65
Strategy	26,548,315.17	27,566,972.91	1,018,657.74	3.84
Operations	31,066,612.97	31,175,675.70	109,062.73	0.35
Analytics	29,884,750.33	27,911,696.96	-1,973,053.37	-6.60

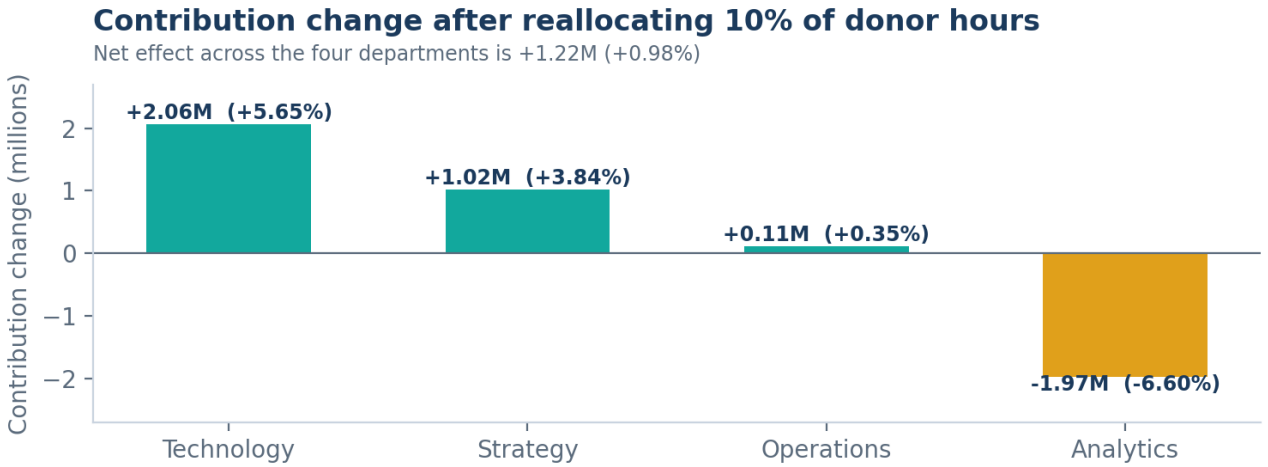


Figure 5.2 — Contribution change by department after reallocating 10% of donor hours (Question 3, Step 5).

Step 6 — Sensitivity analysis

The whole calculation is repeated for three populations through UNION ALL and PARTITION BY scenario: all projects, active employees only, and projects with at least 100 billable hours.

QUESTION 3 · STEP 6 — SENSITIVITY ANALYSIS

```
WITH project_hours AS (
  SELECT
    project_id,
    SUM(billable_hours) AS total_project_hours
  FROM consulting_delivery_performance
  GROUP BY project_id
),
scenario_data AS (
     -- Scenario 1: All projects
     SELECT
       'All projects' AS scenario,
       employee_id,
       project_id,
       department,
       client_tier,
       billable_hours,
       recognized_revenue,
       delivery_cost,
       employee_status
     FROM consulting_delivery_performance
     UNION ALL
     -- Scenario 2: Active employees only
     SELECT
       'Active employees only',
       employee_id,
       project_id,
       department,
       client_tier,
       billable_hours,
       recognized_revenue,
       delivery_cost,
       employee_status
     FROM consulting_delivery_performance
     WHERE employee_status <> 'Exited'
     UNION ALL
     -- Scenario 3: Projects with at least 100 billable hours
     SELECT
       'Projects >= 100 billable hours',
       d.employee_id,
       d.project_id,
       d.department,
       d.client_tier,
       d.billable_hours,
       d.recognized_revenue,
```

```
       d.delivery_cost,
       d.employee_status
     FROM consulting_delivery_performance d
     JOIN project_hours p
       ON d.project_id = p.project_id
     WHERE p.total_project_hours >= 100
),
scenario_counts AS (
     SELECT
       scenario,
       COUNT(DISTINCT project_id) AS eligible_projects,
       COUNT(DISTINCT employee_id) AS eligible_employees
     FROM scenario_data
     GROUP BY scenario
),
segment_performance AS (
     SELECT
       scenario,
       department,
       client_tier,
         SUM(billable_hours) AS billable_hours,
         SUM(recognized_revenue - delivery_cost) AS contribution,
         SUM(recognized_revenue - delivery_cost)
              / NULLIF(SUM(billable_hours), 0) AS contribution_per_hour
     FROM scenario_data
     GROUP BY scenario, department, client_tier
),
ranked_segments AS (
     SELECT
         *,
         ROW_NUMBER() OVER (
              PARTITION BY scenario
              ORDER BY contribution_per_hour
         ) AS rn,
         COUNT(*) OVER (
              PARTITION BY scenario
         ) AS total_segments
     FROM segment_performance
     WHERE contribution_per_hour IS NOT NULL
),
median_value AS (
     SELECT
         scenario,
         AVG(contribution_per_hour) AS median_rate
```

```
    FROM ranked_segments
    WHERE rn IN (
        FLOOR((total_segments + 1) / 2),
        FLOOR((total_segments + 2) / 2)
    )
    GROUP BY scenario
),
classified AS (
    SELECT
     s.*,
     CASE
       WHEN s.contribution_per_hour < m.median_rate
            THEN 'Donor'
       WHEN s.contribution_per_hour > m.median_rate
            THEN 'Recipient'
       ELSE 'Median'
     END AS segment_group
    FROM segment_performance s
    JOIN median_value m
     ON s.scenario = m.scenario
),
reallocation_pool AS (
    SELECT
     scenario,
     SUM(
       CASE
            WHEN segment_group = 'Donor'
            THEN billable_hours * 0.10
            ELSE 0
       END
     ) AS donated_hours,
     SUM(
       CASE
            WHEN segment_group = 'Recipient'
            THEN billable_hours
            ELSE 0
       END
     ) AS recipient_hours
    FROM classified
    GROUP BY scenario
),
scenario_changes AS (
    SELECT
      c.scenario,

      SUM(c.contribution) AS baseline_contribution,
      SUM(
        CASE
           WHEN c.segment_group = 'Donor'
              THEN -c.billable_hours * 0.10
                   * c.contribution_per_hour
           WHEN c.segment_group = 'Recipient'
              THEN p.donated_hours
                   * c.billable_hours
                   / NULLIF(p.recipient_hours, 0)
                   * c.contribution_per_hour
           ELSE 0
        END
      ) AS contribution_change
    FROM classified c
    JOIN reallocation_pool p
      ON c.scenario = p.scenario
    GROUP BY c.scenario
)
SELECT
    sc.scenario,
    sc.eligible_projects,
    sc.eligible_employees,
    ROUND(ch.contribution_change, 2) AS contribution_change,
    ROUND(
   ch.contribution_change
   / NULLIF(ch.baseline_contribution, 0) * 100,
   2
  ) AS change_pct
FROM scenario_counts sc
JOIN scenario_changes ch
  ON sc.scenario = ch.scenario
ORDER BY CASE sc.scenario
  WHEN 'All projects' THEN 1
  WHEN 'Active employees only' THEN 2
  WHEN 'Projects >= 100 billable hours' THEN 3
END;
```

OUTPUT — AS EXECUTED

scenario	eligible_projects	eligible_employees	contribution_change	change_pct
All projects	176	120	1,219,466.47	0.98
Active employees only	173	120	1,011,022.40	0.97
Projects >= 100 billable hours	174	120	1,203,753.83	0.97

KEY INSIGHT

The reallocation is worth about +0.98% of contribution (1,219,466.47), and the estimate barely moves when exited employees (0.97%) or small projects (0.97%) are removed, so the headline is robust. The department picture is not uniform: Technology gains 5.65% and Strategy 3.84% while Analytics loses 6.60% because two of its three segments are donors. Segment classification is also thin at the median — Technology × Strategic (721.05) is a donor and Analytics × Strategic (721.52) a recipient on a gap of 0.24 per hour either side of 721.28.

06
SECTION

Compensation & Promotion Alignment

Question 4: high performers who have not moved up, recent promotees who are below their level's median quality, and how active and exited employees compare.

Question 4 — Compensation and promotion alignment

Identify employees whose productivity and quality remain above peer medians but who have not been promoted in the last three years; identify recently promoted employees whose post-promotion quality is below their job-level median; and compare active and exited employees descriptively.

Step 1 — Promotion recency

Days since promotion against CURDATE() with a 1,095-day threshold. The excerpt was run on 25 October 2026 (an 03-10-2024 promotion shows 752 days). Rows with a blank promotion date are labelled 'Recently promoted', which is Defect 1 in the code review.

QUESTION 4 · STEP 1 — PROMOTION RECENCY

```
SELECT DISTINCT
  employee_id,
  department,
  job_level,
  employee_status,
  promotion_date,
  DATEDIFF(CURDATE(), promotion_date)
    AS days_since_promotion,
  CASE
    WHEN promotion_date IS NULL
      THEN 'No Promotion Date'
    WHEN DATEDIFF(CURDATE(), promotion_date) > 1095
      THEN 'Not promoted in 3 years'
    ELSE 'Recently promoted'
  END AS promotion_status
FROM consulting_delivery_performance
ORDER BY employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	employee_status	promotion_date	days_since_promotion	promotion_status
E0001	Analytics	L4	Exited	03-10-2024	752	Recently promoted
E0001	Operations	L3	Exited	02-05-2025	541	Recently promoted
E0001	Strategy	L1	Active		NULL	Recently promoted
E0001	Strategy	L5	Active	18-07-2025	464	Recently promoted
E0001	Technology	L3	Active	07-09-2025	413	Recently promoted
E0001	Technology	L3	Exited	08-06-2023	1235	Not promoted in 3 years
E0002	Analytics	L5	Active	05-12-2024	689	Recently promoted
E0002	Technology	L4	Active	18-07-2022	1560	Not promoted in 3 years
E0003	Analytics	L3	Active	09-02-2022	1719	Not promoted in 3 years
E0003	Analytics	L6	Active	10-01-2020	2480	Not promoted in 3 years
E0003	Strategy	L2	Active	28-03-2022	1672	Not promoted in 3 years
E0003	Technology	L1	Exited		NULL	Recently promoted
E0003	Technology	L3	Active	21-09-2021	1860	Not promoted in 3 years
E0003	Technology	L3	Active	02-01-2023	1392	Not promoted in 3 years
E0004	Analytics	L1	Active	23-09-2024	762	Recently promoted

Step 2 — Employee relative productivity

Each employee's revenue per billable hour divided by the pooled rate of their department and job level; for Analytics L1 the peer rate is 1,805.72.

QUESTION 4 · STEP 2 — EMPLOYEE RELATIVE PRODUCTIVITY

```sql
WITH employee_performance AS (
  SELECT
    employee_id,
    department,
    job_level,
    employee_status,
    promotion_date,
    SUM(recognized_revenue) AS total_revenue,
    SUM(billable_hours) AS total_billable_hours,
       AVG(quality_score) AS avg_quality_score
     FROM consulting_delivery_performance
     GROUP BY
       employee_id,
       department,
       job_level,
       employee_status,
       promotion_date
),
employee_productivity AS (
     SELECT
       *,
       total_revenue / NULLIF(total_billable_hours, 0)
            AS revenue_per_billable_hour
     FROM employee_performance
),
peer_productivity AS (
     SELECT
       department,
       job_level,
       SUM(total_revenue)
            / NULLIF(SUM(total_billable_hours), 0)
            AS peer_revenue_per_billable_hour
     FROM employee_performance
     GROUP BY department, job_level
)
SELECT
     e.employee_id,
     e.department,
     e.job_level,
     e.employee_status,
  ROUND(e.revenue_per_billable_hour, 2)
    AS revenue_per_billable_hour,
  ROUND(p.peer_revenue_per_billable_hour, 2)
    AS peer_revenue_per_billable_hour,

  ROUND(
    e.revenue_per_billable_hour
    / NULLIF(p.peer_revenue_per_billable_hour, 0),
    2
  ) AS relative_productivity,
  ROUND(e.avg_quality_score, 2) AS avg_quality_score
FROM employee_productivity e
JOIN peer_productivity p
  ON e.department = p.department
 AND e.job_level = p.job_level
ORDER BY e.department, e.job_level, e.employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	employee_status	revenue_per_billable_hour	peer_revenue_per_billable_hour	relative_productivity	avg_quality_score
E0004	Analytics	L1	Active	1637.48	1805.72	0.91	76
E0007	Analytics	L1	Active	1998.88	1805.72	1.11	67
E0010	Analytics	L1	Active	1625.16	1805.72	0.9	93
E0028	Analytics	L1	Active	2025.94	1805.72	1.12	79
E0028	Analytics	L1	Active	1616.91	1805.72	0.9	76
E0028	Analytics	L1	Active	2067.68	1805.72	1.15	100
E0033	Analytics	L1	Active	1813.39	1805.72	1	73
E0042	Analytics	L1	Active	1593.25	1805.72	0.88	76
E0045	Analytics	L1	Active	1061.26	1805.72	0.59	77
E0046	Analytics	L1	Active	806.02	1805.72	0.45	89
E0046	Analytics	L1	Active	2565.05	1805.72	1.42	76
E0046	Analytics	L1	Exited	1902.28	1805.72	1.05	89
E0054	Analytics	L1	Active	2597.99	1805.72	1.44	75
E0065	Analytics	L1	Active	2129.18	1805.72	1.18	77
E0067	Analytics	L1	Active	869.88	1805.72	0.48	84

Step 3 — Peer medians

Median relative productivity and median quality per department and job level, with the size of each peer group (7 to 54 employees in the excerpt).

QUESTION 4 · STEP 3 — PEER MEDIANS

```sql
WITH employee_performance AS (
  SELECT
    employee_id,
    department,
    job_level,
    employee_status,
       promotion_date,
       SUM(recognized_revenue) AS total_revenue,
       SUM(billable_hours) AS total_billable_hours,
       AVG(quality_score) AS avg_quality_score
     FROM consulting_delivery_performance
     GROUP BY
      employee_id,
       department,
       job_level,
      employee_status,
       promotion_date
),
employee_productivity AS (
     SELECT
      *,
       total_revenue / NULLIF(total_billable_hours, 0)
           AS revenue_per_billable_hour
     FROM employee_performance
),
peer_productivity AS (
     SELECT
       department,
       job_level,
       SUM(total_revenue)
           / NULLIF(SUM(total_billable_hours), 0)
           AS peer_revenue_per_billable_hour
     FROM employee_performance
     GROUP BY department, job_level
),
employee_relative AS (
     SELECT
       e.*,
       e.revenue_per_billable_hour
         / NULLIF(p.peer_revenue_per_billable_hour, 0)
         AS relative_productivity
     FROM employee_productivity e
     JOIN peer_productivity p
      ON e.department = p.department
```

```
    AND e.job_level = p.job_level
),
ranked_peers AS (
    SELECT
      department,
      job_level,
      relative_productivity,
      avg_quality_score,
      ROW_NUMBER() OVER (
        PARTITION BY department, job_level
        ORDER BY relative_productivity
      ) AS productivity_rank,
      ROW_NUMBER() OVER (
        PARTITION BY department, job_level
        ORDER BY avg_quality_score
      ) AS quality_rank,
      COUNT(*) OVER (
        PARTITION BY department, job_level
      ) AS peer_count
    FROM employee_relative
)
SELECT
    department,
    job_level,
  ROUND(AVG(
    CASE
      WHEN productivity_rank IN (
          FLOOR((peer_count + 1) / 2),
          FLOOR((peer_count + 2) / 2)
      )
      THEN relative_productivity
    END
  ), 2) AS median_relative_productivity,
  ROUND(AVG(
    CASE
      WHEN quality_rank IN (
          FLOOR((peer_count + 1) / 2),
          FLOOR((peer_count + 2) / 2)
      )
      THEN avg_quality_score
    END
  ), 2) AS median_quality_score,
  MAX(peer_count) AS employees_in_peer_group
FROM ranked_peers

GROUP BY department, job_level
ORDER BY department, job_level;
```

OUTPUT — FIRST ROWS AS EXECUTED

department	job_level	median_relative_productivity	median_quality_score	employees_in_peer_group
Analytics	L1	1	79	27
Analytics	L2	1.03	82	37
Analytics	L3	1.06	83.5	54
Analytics	L4	0.92	85	44
Analytics	L5	0.96	81.5	26
Analytics	L6	1.2	78	7
Operations	L1	0.87	77	22
Operations	L2	1.07	82.5	34
Operations	L3	0.92	79	45
Operations	L4	0.94	79	36
Operations	L5	0.96	80	28
Operations	L6	0.97	87	11
Strategy	L1	0.96	83	18
Strategy	L2	1.05	78	25
Strategy	L3	1.02	84	52

Step 4 — Employees for promotion review

Above-median productivity and above-median quality, with no promotion in more than three years (or no promotion date). The excerpt lists consultants such as E0040 (Analytics L2, relative productivity 1.55, quality 100, 2,207 days since promotion) and E0101 (1.44, quality 93, 2,322 days).

QUESTION 4 · STEP 4 — EMPLOYEES FOR PROMOTION REVIEW

```
WITH employee_performance AS (
     SELECT
      employee_id,
      department,
      job_level,
      employee_status,
      promotion_date,
      SUM(recognized_revenue) AS total_revenue,
      SUM(billable_hours) AS total_billable_hours,
      AVG(quality_score) AS quality_score
     FROM consulting_delivery_performance
     GROUP BY
      employee_id,
      department,
      job_level,
      employee_status,
      promotion_date
),
employee_productivity AS (
     SELECT
       *,
       total_revenue / NULLIF(total_billable_hours, 0)
            AS revenue_per_billable_hour
     FROM employee_performance
),
peer_productivity AS (
     SELECT
       department,
       job_level,
       SUM(total_revenue)
            / NULLIF(SUM(total_billable_hours), 0)
            AS peer_revenue_per_billable_hour
     FROM employee_performance
     GROUP BY department, job_level
),
employee_relative AS (
     SELECT
       e.*,
       e.revenue_per_billable_hour
            / NULLIF(p.peer_revenue_per_billable_hour, 0)
            AS relative_productivity
     FROM employee_productivity e
     JOIN peer_productivity p
      ON e.department = p.department
```

```
     AND e.job_level = p.job_level
),
ranked_peers AS (
     SELECT
       *,
      ROW_NUMBER() OVER (
        PARTITION BY department, job_level
        ORDER BY relative_productivity
      ) AS productivity_rank,
      ROW_NUMBER() OVER (
        PARTITION BY department, job_level
        ORDER BY quality_score
      ) AS quality_rank,
      COUNT(*) OVER (
        PARTITION BY department, job_level
      ) AS peer_count
     FROM employee_relative
),
peer_medians AS (
     SELECT
      department,
      job_level,
      AVG(
        CASE
          WHEN productivity_rank IN (
              FLOOR((peer_count + 1) / 2),
              FLOOR((peer_count + 2) / 2)
          )
          THEN relative_productivity
        END
      ) AS median_productivity,
      AVG(
        CASE
          WHEN quality_rank IN (
              FLOOR((peer_count + 1) / 2),
              FLOOR((peer_count + 2) / 2)
              )
              THEN quality_score
            END
          ) AS median_quality
        FROM ranked_peers
        GROUP BY department, job_level
)
SELECT

        e.employee_id,
        e.department,
        e.job_level,
        ROUND(e.relative_productivity, 2)
          AS relative_productivity,
        ROUND(e.quality_score, 2) AS quality_score,
        DATEDIFF(CURDATE(), e.promotion_date)
          AS days_since_promotion,
        'Review' AS promotion_review_flag
FROM employee_relative e
JOIN peer_medians m
    ON e.department = m.department
AND e.job_level = m.job_level
WHERE e.relative_productivity > m.median_productivity
    AND e.quality_score > m.median_quality
    AND (
         e.promotion_date IS NULL
         OR DATEDIFF(CURDATE(), e.promotion_date) > 1095
    )
ORDER BY e.department, e.job_level, e.employee_id;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	relative_productivity	quality_score	days_since_promotion	promotion_review_flag
E0046	Analytics	L1	1.05	89	2194	Review
E0069	Analytics	L1	1.23	94	1657	Review
E0118	Analytics	L1	1.32	94	1263	Review
E0118	Analytics	L1	1.12	87	1131	Review
E0006	Analytics	L2	1.06	100	1469	Review
E0015	Analytics	L2	1.35	85	1436	Review
E0040	Analytics	L2	1.55	100	2207	Review
E0040	Analytics	L2	1.55	95	1710	Review
E0058	Analytics	L2	1.12	92	2248	Review
E0074	Analytics	L2	1.08	93	1604	Review
E0101	Analytics	L2	1.44	93	2322	Review
E0104	Analytics	L2	1.23	89	1812	Review
E0008	Analytics	L3	1.08	89	1715	Review
E0027	Analytics	L3	1.36	95	1832	Review
E0044	Analytics	L3	1.11	95	2429	Review

Step 5 — Recently promoted, below-median quality

Promoted within three years but below the job-level median quality, ordered by level then lowest quality. E0109 (L2, quality 60) and E0035 (L2, quality 61) are the lowest L2 rows in the excerpt.

QUESTION 4 · STEP 5 — RECENTLY PROMOTED, BELOW-MEDIAN QUALITY

```
WITH employee_performance AS (
     SELECT
      employee_id,
      department,
      job_level,
      employee_status,
      promotion_date,
      AVG(quality_score) AS quality_score
     FROM consulting_delivery_performance
     GROUP BY
      employee_id,
      department,
      job_level,
      employee_status,
      promotion_date
),
ranked_quality AS (
     SELECT
      *,
       ROW_NUMBER() OVER (
         PARTITION BY job_level
         ORDER BY quality_score
       ) AS quality_rank,
       COUNT(*) OVER (
         PARTITION BY job_level
       ) AS peer_count
     FROM employee_performance
),
job_level_medians AS (
     SELECT
       job_level,
       AVG(
         CASE
           WHEN quality_rank IN (
               FLOOR((peer_count + 1) / 2),
               FLOOR((peer_count + 2) / 2)
           )
           THEN quality_score
         END
       ) AS median_quality
     FROM ranked_quality
     GROUP BY job_level
)
SELECT
```

```
    e.employee_id,
    e.department,
    e.job_level,
    ROUND(e.quality_score, 2) AS quality_score,
    ROUND(m.median_quality, 2) AS job_level_median_quality,
    DATEDIFF(CURDATE(), e.promotion_date)
   AS days_since_promotion,
  e.employee_status
FROM employee_performance e
JOIN job_level_medians m
  ON e.job_level = m.job_level
WHERE e.promotion_date IS NOT NULL
 AND DATEDIFF(CURDATE(), e.promotion_date) <= 1095
 AND DATEDIFF(CURDATE(), e.promotion_date) >= 0
 AND e.quality_score < m.median_quality
ORDER BY e.job_level, e.quality_score;
```

OUTPUT — FIRST ROWS AS EXECUTED

employee_id	department	job_level	quality_score	job_level_median_quality	days_since_promotion	employee_status
E0111	Analytics	L1	68	80	995	Active
E0052	Technology	L1	75	80	1012	Exited
E0004	Analytics	L1	76	80	762	Active
E0028	Analytics	L1	76	80	672	Active
E0046	Analytics	L1	76	80	403	Active
E0089	Analytics	L1	77	80	459	Active
E0093	Technology	L1	77	80	817	Active
E0069	Strategy	L1	77	80	466	Active
E0091	Analytics	L1	79	80	613	Active
E0109	Strategy	L2	60	81	413	Active
E0035	Analytics	L2	61	81	686	Active
E0114	Technology	L2	66	81	539	Active
E0086	Strategy	L2	67	81	832	Active
E0026	Operations	L2	68	81	489	Exited
E0070	Technology	L2	69	81	929	Active

Step 6 — Active versus exited employees

A descriptive comparison with no causal claim.

QUESTION 4 · STEP 6 — ACTIVE VERSUS EXITED EMPLOYEES

```sql
WITH employee_performance AS (
  SELECT
    employee_id,
    department,
    job_level,
    employee_status,
    promotion_date,
    SUM(recognized_revenue) AS total_revenue,
    SUM(billable_hours) AS total_billable_hours,
       AVG(quality_score) AS quality_score
     FROM consulting_delivery_performance
     GROUP BY
      employee_id,
       department,
       job_level,
      employee_status,
       promotion_date
),
employee_productivity AS (
     SELECT
      *,
       total_revenue / NULLIF(total_billable_hours, 0)
           AS revenue_per_billable_hour
     FROM employee_performance
),
peer_productivity AS (
     SELECT
       department,
       job_level,
       SUM(total_revenue)
           / NULLIF(SUM(total_billable_hours), 0)
           AS peer_revenue_per_billable_hour
     FROM employee_performance
     GROUP BY department, job_level
),
employee_relative AS (
     SELECT
      e.*,
      e.revenue_per_billable_hour
           / NULLIF(p.peer_revenue_per_billable_hour, 0)
        AS relative_productivity
    FROM employee_productivity e
    JOIN peer_productivity p
     ON e.department = p.department

    AND e.job_level = p.job_level
)
SELECT
    employee_status,
    COUNT(*) AS employee_count,
    SUM(
      CASE
        WHEN relative_productivity > 1
         AND quality_score > 0
         AND (
             promotion_date IS NULL
             OR DATEDIFF(CURDATE(), promotion_date) > 1095
         )
        THEN 1
        ELSE 0
      END
    ) AS high_productivity_not_recently_promoted,
    ROUND(AVG(quality_score), 2) AS average_quality_score,
    ROUND(AVG(relative_productivity), 2)
      AS average_relative_productivity
FROM employee_relative
GROUP BY employee_status
ORDER BY employee_status;
```

OUTPUT — AS EXECUTED

employee_status	employee_count	high_productivity_not_recently_promoted	average_quality_score	average_relative_productivity
Active	599	150	81.97	1.01
Exited	98	25	82.80	1.03

KEY INSIGHT

Roughly a quarter of both groups are above-peer performers with no promotion in three years: 150 of 599 active (25.0%) and 25 of 98 exited (25.5%). Exited employees were not weaker — their average quality (82.80) and relative productivity (1.03) are slightly higher than active ones (81.97 and 1.01) — which makes the promotion-lag pool a plausible retention risk rather than a performance problem. The counts are employee-department-level groups, not people (Defect 3).

07
SECTION

Executive Decision Under Uncertainty

Question 5: which departments to act on first, and which of those calls depend on the benchmark chosen.

Question 5 — Executive decision under uncertainty

Prioritise resource-allocation risks, quantify classification changes under alternative peer groups, and separate robust findings from benchmark-sensitive ones.

Step 1 — Department risk metrics

Quality, rework, productivity, cost and contribution by department.

QUESTION 5 · STEP 1 — DEPARTMENT RISK METRICS

```sql
SELECT
  department,
  COUNT(*) AS total_records,
  ROUND(AVG(quality_score), 2)
    AS average_quality_score,
  ROUND(
    SUM(rework_hours) * 100.0
    / NULLIF(
        SUM(billable_hours + non_billable_hours), 0
    ),
    2
  ) AS rework_rate_pct,
  ROUND(
    SUM(recognized_revenue)
    / NULLIF(SUM(billable_hours), 0),
    2
  ) AS revenue_per_billable_hour,
  ROUND(SUM(delivery_cost), 2)
    AS total_delivery_cost,
  ROUND(
    SUM(recognized_revenue - delivery_cost), 2
  ) AS total_contribution
FROM consulting_delivery_performance
GROUP BY department
ORDER BY total_contribution DESC;
```

OUTPUT — AS EXECUTED

department	total_records	avg_quality_score	rework_rate_pct	revenue_per_billable_hour	total_delivery_cost	total_contribution
Technology	168	82.49	10.05	1,745.68	37,550,640.63	36,576,287.10
Operations	176	81.27	10.05	1,647.53	41,993,057.41	31,066,612.97
Analytics	196	82.43	10.57	1,627.67	42,497,947.05	29,884,750.33
Strategy	160	82.16	10.06	1,636.75	36,787,258.01	26,548,315.17

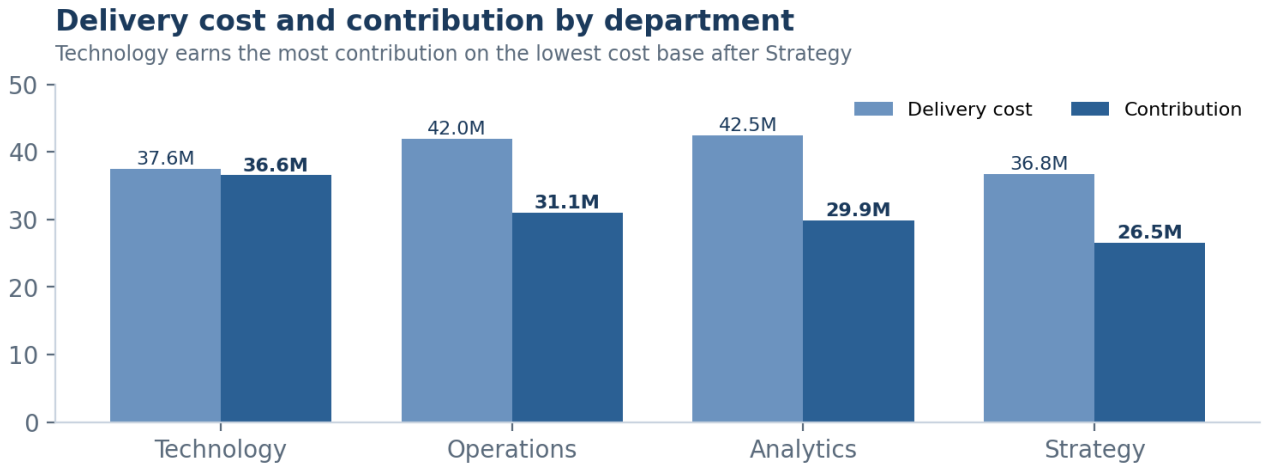


Figure 7.1 — Delivery cost and contribution by department (Question 5, Step 1).

Step 2 — Department risk segments

Rule order: high rework with low quality, then low relative productivity, then high delivery cost. Department rework sits at 10.05 to 10.57%, but no department's average quality is below 80, so the first rule cannot fire at this level.

QUESTION 5 · STEP 2 — DEPARTMENT RISK SEGMENTS

```sql
WITH department_metrics AS (
     SELECT
      department,
      COUNT(*) AS total_records,
      AVG(quality_score) AS average_quality_score,
      SUM(rework_hours) * 100.0
        / NULLIF(
          SUM(billable_hours + non_billable_hours), 0
        ) AS rework_rate_pct,
      SUM(recognized_revenue)
        / NULLIF(SUM(billable_hours), 0)
        AS revenue_per_billable_hour,
      SUM(delivery_cost) AS total_delivery_cost
     FROM consulting_delivery_performance
     GROUP BY department
),
overall_metrics AS (
     SELECT
      AVG(revenue_per_billable_hour)
        AS overall_productivity,
      AVG(total_delivery_cost)
        AS average_delivery_cost
     FROM department_metrics
)
SELECT
  d.department,
  ROUND(d.average_quality_score, 2)
    AS average_quality_score,
  ROUND(d.rework_rate_pct, 2)
    AS rework_rate_pct,
  ROUND(d.revenue_per_billable_hour, 2)
    AS revenue_per_billable_hour,
  ROUND(d.total_delivery_cost, 2)
    AS total_delivery_cost,
  CASE
    WHEN d.rework_rate_pct >= 10
       AND d.average_quality_score < 80
      THEN 'High rework, low quality'
    WHEN d.revenue_per_billable_hour
       < o.overall_productivity
      THEN 'Low relative productivity'
    WHEN d.total_delivery_cost
       > o.average_delivery_cost
      THEN 'High delivery cost'
```

```
    ELSE 'No major risk flagged'
  END AS risk_segment
FROM department_metrics d
CROSS JOIN overall_metrics o
ORDER BY d.department;
```

OUTPUT — AS EXECUTED

department	avg_quality_score	rework_rate_pct	revenue_per_billable_hour	total_delivery_cost	risk_segment
Analytics	82.43	10.57	1,627.67	42,497,947.05	Low relative productivity
Operations	81.27	10.05	1,647.53	41,993,057.41	Low relative productivity
Strategy	82.16	10.06	1,636.75	36,787,258.01	Low relative productivity
Technology	82.49	10.05	1,745.68	37,550,640.63	No major risk flagged

Step 3 — Records affected by each risk

The same three rules applied to individual records against their own department's averages, with the contribution those records carry.

QUESTION 5 · STEP 3 — RECORDS AFFECTED BY EACH RISK

```
WITH record_metrics AS (
     SELECT
      record_id,
      employee_id,
      department,
      job_level,
      billable_hours,
      non_billable_hours,
      recognized_revenue,
      delivery_cost,
      quality_score,
      rework_hours,
      recognized_revenue - delivery_cost AS contribution,
      rework_hours * 100.0
        / NULLIF(billable_hours + non_billable_hours, 0)
        AS rework_rate_pct,
      recognized_revenue
        / NULLIF(billable_hours, 0)
        AS revenue_per_billable_hour
     FROM consulting_delivery_performance
),
department_benchmarks AS (
     SELECT
      department,
      AVG(revenue_per_billable_hour) AS avg_productivity,
      AVG(delivery_cost) AS avg_delivery_cost
     FROM record_metrics
     GROUP BY department
),
risk_records AS (
     SELECT
      r.*,
      CASE
        WHEN r.rework_rate_pct >= 10
          AND r.quality_score < 80
          THEN 'High rework, low quality'
        WHEN r.revenue_per_billable_hour < b.avg_productivity
          THEN 'Low relative productivity'
        WHEN r.delivery_cost > b.avg_delivery_cost
          THEN 'High delivery cost'
        ELSE 'No major risk flagged'
      END AS risk_segment
    FROM record_metrics r
    JOIN department_benchmarks b
```

```
      ON r.department = b.department
)
SELECT
    department,
    risk_segment,
    COUNT(*) AS affected_records,
    ROUND(SUM(contribution), 2)
      AS estimated_contribution_opportunity
FROM risk_records
WHERE risk_segment <> 'No major risk flagged'
GROUP BY department, risk_segment
ORDER BY affected_records DESC;
```

OUTPUT — AS EXECUTED

department	risk_segment	affected_records	estimated_contribution_opportunity
Analytics	Low relative productivity	73	3,693,159.01
Operations	Low relative productivity	72	4,490,891.97
Strategy	Low relative productivity	67	4,227,140.85
Technology	Low relative productivity	65	6,313,534.29
Analytics	High rework, low quality	44	4,103,864.96
Operations	High rework, low quality	43	7,867,940.35
Analytics	High delivery cost	37	14,978,639.82
Strategy	High rework, low quality	35	5,666,896.60
Operations	High delivery cost	34	12,729,999.22
Technology	High rework, low quality	33	4,374,681.83
Technology	High delivery cost	27	13,526,505.98
Strategy	High delivery cost	24	10,230,796.92

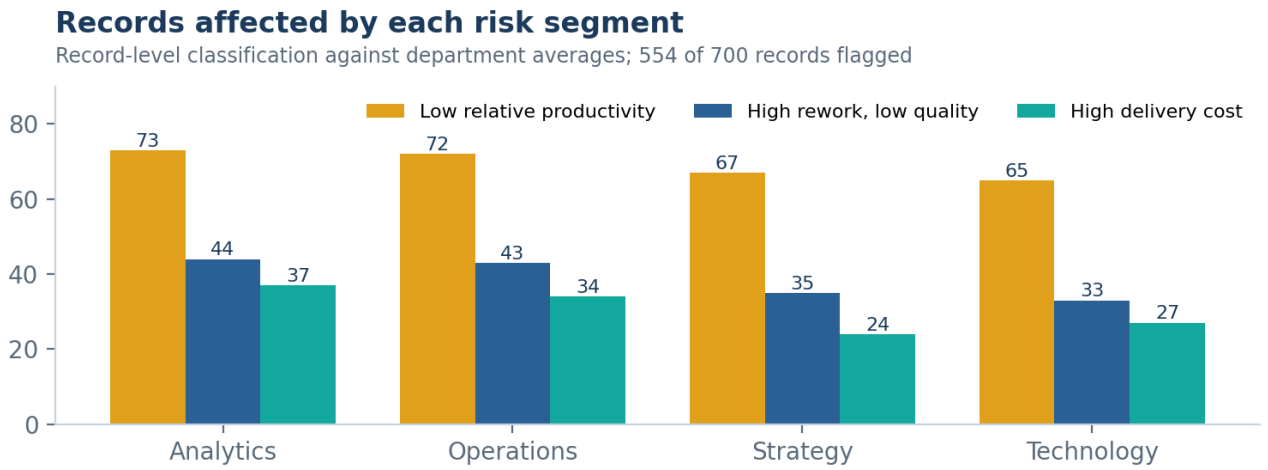


Figure 7.2 — Records affected by each risk segment (Question 5, Step 3).

Step 4 — Risk classification under alternative benchmarks

Low-productivity classification is recomputed against the company average and against each department's own average.

QUESTION 5 · STEP 4 — RISK CLASSIFICATION UNDER ALTERNATIVE BENCHMARKS

```sql
WITH record_metrics AS (
  SELECT
    record_id,
    department,
    billable_hours,
    non_billable_hours,
    recognized_revenue,
    delivery_cost,
    quality_score,
    rework_hours,
    employee_status,
    recognized_revenue - delivery_cost AS contribution,
    rework_hours * 100.0
      / NULLIF(billable_hours + non_billable_hours, 0)
      AS rework_rate_pct,
    recognized_revenue
      / NULLIF(billable_hours, 0)
      AS revenue_per_billable_hour
  FROM consulting_delivery_performance
),
benchmarks AS (
     SELECT
       AVG(revenue_per_billable_hour) AS overall_productivity
     FROM record_metrics
),
comparison AS (
     SELECT
       r.*,
       CASE
         WHEN r.revenue_per_billable_hour < b.overall_productivity
              THEN 'Low relative productivity'
         ELSE 'Other'
       END AS company_benchmark,
       CASE
         WHEN r.revenue_per_billable_hour <
              (
                  SELECT AVG(r2.revenue_per_billable_hour)
                  FROM record_metrics r2
                  WHERE r2.department = r.department
              )
              THEN 'Low relative productivity'
         ELSE 'Other'
       END AS department_benchmark
     FROM record_metrics r

     CROSS JOIN benchmarks b
)
SELECT
     department,
     COUNT(*) AS total_records,
     SUM(
    company_benchmark = 'Low relative productivity'
  ) AS company_benchmark_risk_records,
  SUM(
    department_benchmark = 'Low relative productivity'
  ) AS department_benchmark_risk_records,
  SUM(
    company_benchmark <> department_benchmark
  ) AS changed_classifications,
  ROUND(
    SUM(
         CASE
          WHEN company_benchmark <> department_benchmark
               THEN contribution
          ELSE 0
         END
    ),
    2
  ) AS contribution_in_changed_records
FROM comparison
GROUP BY department
ORDER BY changed_classifications DESC;
```

OUTPUT — AS EXECUTED

department	total_records	company_benchmark_risk_records	department_benchmark_risk_records	changed_classifications	contribution_in_changed_records
Operations	176	94	92	2	34,811.15
Technology	168	78	80	2	486,016.10
Analytics	196	96	95	1	69,681.80
Strategy	160	81	81	0	0

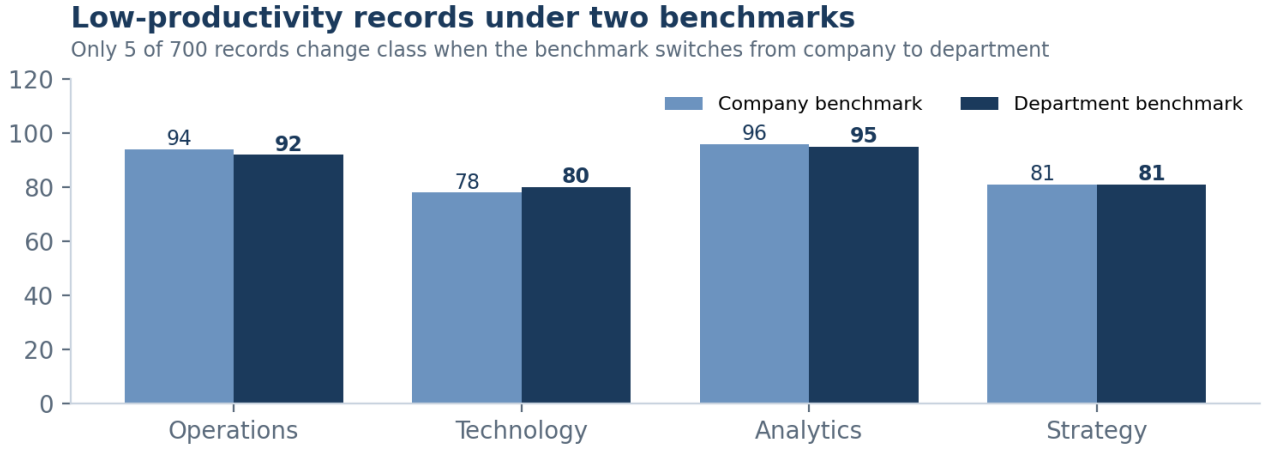


Figure 7.3 — Low-productivity records under company and department benchmarks (Question 5, Step 4).

Step 5 — Executive prioritisation summary

Departments ranked by the contribution attached to their risk label, with a sensitivity band from the benchmark swap in Step 4.

QUESTION 5 · STEP 5 — EXECUTIVE PRIORITISATION SUMMARY

```
WITH record_metrics AS (
  SELECT
    record_id,
    department,
      billable_hours,
      non_billable_hours,
      recognized_revenue,
      delivery_cost,
      quality_score,
      rework_hours,
      employee_status,
      recognized_revenue - delivery_cost AS contribution,
      rework_hours * 100.0
        / NULLIF(billable_hours + non_billable_hours, 0)
        AS rework_rate_pct,
      recognized_revenue
        / NULLIF(billable_hours, 0)
        AS revenue_per_billable_hour
     FROM consulting_delivery_performance
),
department_metrics AS (
     SELECT
      department,
      AVG(quality_score) AS avg_quality,
      AVG(rework_rate_pct) AS avg_rework_rate,
      AVG(revenue_per_billable_hour) AS avg_productivity,
      AVG(delivery_cost) AS avg_delivery_cost
     FROM record_metrics
     GROUP BY department
),
department_risks AS (
     SELECT
      department,
      CASE
        WHEN avg_rework_rate >= 10 AND avg_quality < 80
             THEN 'High rework, low quality'
         WHEN avg_productivity < (
             SELECT AVG(avg_productivity)
             FROM department_metrics
         )
             THEN 'Low relative productivity'
         WHEN avg_delivery_cost > (
             SELECT AVG(avg_delivery_cost)
             FROM department_metrics
```

```
        )
            THEN 'High delivery cost'
        ELSE 'No major risk flagged'
      END AS risk_segment
    FROM department_metrics
),
risk_summary AS (
    SELECT
      r.department,
      d.risk_segment,
      COUNT(*) AS affected_records,
      ROUND(SUM(r.contribution), 2)
        AS estimated_contribution_opportunity
    FROM record_metrics r
    JOIN department_risks d
      ON r.department = d.department
    WHERE d.risk_segment <> 'No major risk flagged'
    GROUP BY r.department, d.risk_segment
),
sensitivity AS (
    SELECT
      department,
     SUM(
        revenue_per_billable_hour < (
            SELECT AVG(revenue_per_billable_hour)
            FROM record_metrics
        )
     ) AS company_risk_records,
     SUM(
        revenue_per_billable_hour < (
            SELECT AVG(r2.revenue_per_billable_hour)
            FROM record_metrics r2
            WHERE r2.department = record_metrics.department
        )
     ) AS department_risk_records
    FROM record_metrics
    GROUP BY department
)
SELECT
    ROW_NUMBER() OVER (
      ORDER BY rs.estimated_contribution_opportunity DESC
    ) AS priority,
    rs.department,
    rs.risk_segment,

    rs.affected_records,
    rs.estimated_contribution_opportunity,
    CASE
      WHEN ABS(s.company_risk_records - s.department_risk_records) = 0
        THEN 'Low'
      WHEN ABS(s.company_risk_records - s.department_risk_records)
        <= 5
        THEN 'Moderate'
      ELSE 'High'
  END AS sensitivity_status
FROM risk_summary rs
JOIN sensitivity s
  ON rs.department = s.department
ORDER BY priority;
```

OUTPUT — AS EXECUTED

priority	department	risk_segment	affected_records	estimated_contribution_opportunity	sensitivity_status
1	Operations	Low relative productivity	176	31,066,612.97	Moderate
2	Analytics	Low relative productivity	196	29,884,750.33	Moderate
3	Strategy	High delivery cost	160	26,548,315.17	Low

KEY INSIGHT

Only 5 of 700 records change class when the productivity benchmark moves from the company to the department (Operations 2, Technology 2, Analytics 1, Strategy 0), so the low-productivity finding is robust. The priority ranking is benchmark-light but scale-heavy: Operations ranks first because it holds the largest contribution of the three flagged departments, and Technology — the strongest department on revenue per billable hour (1,745.68) — is the only one with no department-level flag.

08
SECTION

Code Review & Corrections

Six data and logic defects and six design observations found by reading the queries against their own output, each with a correction.

Reading the queries next to what they returned shows where the data or the logic does not do what its label says. Each finding below is traceable to a visible row, figure or definition in the executed output.

Defect 1 — Blank promotion dates are not NULL

DATA INTEGRITY · STATUS: DOCUMENTED

In Q4 Step 1 the row for E0001 in Strategy L1 has a blank promotion_date and NULL days_since_promotion, yet its status is 'Recently promoted'. The CASE tests promotion_date IS NULL first and would have returned 'No Promotion Date', so the blank value cannot be a true NULL (a zero-date or empty value is the likely cause). The same test is used in Steps 4 and 6, so never-promoted employees drop out of the promotion-review list and of the 'not recently promoted' count.

CORRECTION — DEFECT 1

```sql
-- Detect: blank dates that are not NULL
SELECT COUNT(*) AS blank_not_null
FROM consulting_delivery_performance
WHERE promotion_date IS NOT NULL
  AND DATEDIFF(CURDATE(), promotion_date) IS NULL;

-- Fix: store missing dates as NULL (adjust the literal to what the load produced)
UPDATE consulting_delivery_performance
SET promotion_date = NULL
WHERE promotion_date = '0000-00-00';
```

Defect 2 — CURDATE() makes the promotion results irreproducible

LOGIC · STATUS: DOCUMENTED

Every days_since_promotion value depends on the day the query runs; the same script re-run next month moves employees across the 1,095-day line. The excerpt matches a run on 25 October 2026.

CORRECTION — DEFECT 2

```sql
-- Fix: fix the as-of date once and reuse it
WITH as_of AS (SELECT DATE('2026-10-25') AS report_date)
SELECT DISTINCT d.employee_id,
       DATEDIFF(a.report_date, d.promotion_date) AS days_since_promotion
FROM consulting_delivery_performance d
CROSS JOIN as_of a;
```

Defect 3 — Employee identity is not stable across rows

DATA INTEGRITY · STATUS: NEEDS SOURCE CHECK

The same employee_id appears in several departments and job levels, and with both statuses: E0001 is Exited in Analytics L4 and Operations L3 but Active in Strategy L5 and Technology L3. Q4 Step 6 therefore counts 697 employee–department–level–status groups (599 Active, 98 Exited) for 120 distinct employees, and the 'Active employees only' scenario in Q3 still reports 120 eligible employees. The source should say whether employee_status is a property of the person or of the assignment.

CORRECTION — DEFECT 3

```sql
-- Detect: employees carrying more than one status
SELECT employee_id, COUNT(DISTINCT employee_status) AS statuses
FROM consulting_delivery_performance
GROUP BY employee_id
HAVING COUNT(DISTINCT employee_status) > 1;
```

Defect 4 — 'Three consecutive declines' counts changes, not projects

LOGIC · **STATUS: DOCUMENTED**

Q2 Step 1 counts decline events, so a streak of 3 means four consecutive project records with falling quality (E0010 and E0120, with four declines, span five records). The brief says 'at least three consecutive project records'; if that is the intent, the threshold should be 2 declines, which would add every employee with exactly two.

CORRECTION — DEFECT 4

```
-- Use the intended definition explicitly
HAVING MAX(consecutive_quality_declines) >= 2   -- three consecutive records
-- or keep >= 3 and rename the column to consecutive_decline_events
```

Defect 5 — 'Estimated contribution opportunity' is total contribution

LOGIC · **STATUS: DOCUMENTED**

In Q5 Step 5 the value for Operations (31,066,612.97) equals Operations' total contribution from Step 1, and affected_records (176) is every Operations record. The column ranks departments by size, not by opportunity, so Operations ranks first because it is larger than Strategy, not because more value is at stake.

CORRECTION — DEFECT 5

```
-- Fix: size the opportunity as the gap to the benchmark
-- (hours x (benchmark rate - actual rate))
SELECT department,
       SUM(billable_hours * (b.benchmark_rate
           - recognized_revenue / NULLIF(billable_hours, 0))) AS productivity_gap_value
FROM record_metrics r JOIN benchmark b USING (department)
WHERE recognized_revenue / NULLIF(billable_hours, 0) < b.benchmark_rate
GROUP BY department;
```

Defect 6 — Q5 Steps 2 and 5 label Strategy differently

LOGIC · **STATUS: DOCUMENTED**

Step 2 pools revenue and hours and flags Strategy as 'Low relative productivity'; Step 5 averages record-level ratios and flags it as 'High delivery cost'. Both are defensible, but they are different bases, and the executive summary silently uses the second. Choose one basis and apply it in every step.

Design observations

Observation 7 — Peer groups of one. Q1 Steps 2 and 4 partition by department, level, client tier and complexity, so many groups hold a single employee and rank 1 is automatic. Add COUNT(*) OVER (PARTITION BY ...) as peer_group_size and rank only groups of three or more.

Observation 8 — Linear reallocation model. Q3 assumes the extra hours earn each recipient's current contribution per hour, with no capacity, demand or skill constraint. Treat +0.98% as an upper-bound estimate. Donors are all six below-median segments, not only the bottom-performing ones.

Observation 9 — Razor-thin median split. Technology × Strategic (721.05) and Analytics × Strategic (721.52) are classified on a difference of about 0.03% from the median. Add a neutral band (for example ±2%) so near-median segments neither donate nor receive.

Observation 10 — Non-informative rework rule. Department rework rates are 10.05 to 10.57%, all above the 10% threshold, and no average quality is below 80, so the 'High rework, low quality' rule never discriminates at department level. Calibrate thresholds to percentiles of the data.

Observation 11 — Always-true predicate. Q4 Step 6 includes quality_score > 0, which is true for every row, and defines high productivity without any quality condition, unlike Step 4. Remove it or replace it with the above-median-quality test.

Observation 12 — Arbitrary sensitivity bands. Q5 Step 5 labels a change of 0 as Low, up to 5 as Moderate and above 5 as High. With 160 to 196 records per department, two records is about 1%; express the band as a percentage of department records.

Review summary

Finding	Severity	Status	Action
Blank promotion dates are not NULL	Data integrity	Documented	Convert to NULL
CURDATE() in promotion logic	Logic	Documented	Fixed as-of date
employee_id has several statuses and departments	Data integrity	Source check	Define employee grain
Decline streak counts events	Logic	Documented	Align threshold to brief
Opportunity equals total contribution	Logic	Documented	Use benchmark gap
Strategy classified two ways	Logic	Documented	One basis throughout
Single-member peer groups	Design	Roadmap	Add peer_group_size
Linear reallocation, no capacity	Design	Roadmap	Cap recipient hours
Median split on a 0.03% gap	Design	Roadmap	Neutral band
Rework rule never discriminates	Design	Roadmap	Percentile thresholds
quality_score > 0 predicate	Cosmetic	Documented	Remove
Sensitivity bands use record counts	Design	Roadmap	Use percentages

09
SECTION

Results & Roadmap

What the dataset says, how to rerun the project, and where it goes next.

Key findings from the dataset

Question	Answer from the data
Highest-contribution department	Technology — 36,576,287.10 contribution at 1,745.68 revenue per billable hour; the only department with no flag
Best and worst segment	Technology × Enterprise at 933.44 contribution per billable hour; Strategy × Mid-Market at 559.02
Reallocation upside	+1,219,466.47 (+0.98%); 0.97% when exited employees or sub-100-hour projects are excluded
Departments that gain or lose	Technology +5.65%, Strategy +3.84%, Operations +0.35%, Analytics −6.60%
Longest quality decline	E0010 and E0120 — four consecutive declines
Promotion lag	150 of 599 active and 25 of 98 exited groups: above-peer performance, no promotion in three years
Benchmark sensitivity	5 of 700 records change class between company and department benchmarks
Top executive priority	Operations (Low relative productivity), then Analytics, then Strategy (High delivery cost)

Running the project

SETUP — COMMAND LINE

```
# 1. Load consulting_delivery_performance into MySQL 8.0 (Table Data Import Wizard or LOAD DATA)
mysql -u root -p -e "CREATE DATABASE case_study_2;"

# 2. Run the analytical script
mysql -u root -p case_study_2 < consulting_delivery_analytics.sql

# 3. Verify the load
mysql -u root -p -e "USE case_study_2; SELECT COUNT(*) FROM consulting_delivery_performance;"
# expected: 700
```

Suggested repository structure

REPOSITORY LAYOUT

```
consulting-delivery-analytics/
├── README.md                    # overview, setup, headline results
├── data/
│   └── consulting_delivery_performance.csv
├── sql/
│   ├── 01_setup.sql             # CREATE DATABASE, USE, SHOW TABLES
│   ├── 02_q1_productivity.sql   # Q1 steps 1-4
│   ├── 03_q2_trajectory.sql     # Q2 steps 1-6
│   ├── 04_q3_reallocation.sql   # Q3 steps 1-6
│   ├── 05_q4_promotion.sql      # Q4 steps 1-6
│   ├── 06_q5_executive.sql      # Q5 steps 1-5
│   └── 07_fixes.sql             # corrections from Section 08
├── docs/
│   └── project-documentation.pdf  # this document
└── screenshots/                 # query output from MySQL Workbench
```

Roadmap

Enhancement	Detail
Fix the data defects	Convert blank promotion dates to NULL, define the employee grain, and fix the as-of date (Defects 1 to 3).

Enhancement	Detail
Views	Create employee_metrics and segment_performance views so the repeated CTEs in Questions 3 to 5 read as short statements.
Capacity-aware reallocation	Cap recipient hours by demand and skills, and add a neutral band around the median.
Peer-group size controls	Require a minimum group size before ranking and report it beside every peer rank.
Indexing	Composite indexes on (employee_id, project_start_date) and (department, client_tier); compare EXPLAIN before and after.
Visual layer	Point Power BI or Tableau at the segment and department views for an interactive delivery dashboard.

Skills demonstrated

Peer-group benchmarking with window functions, gaps-and-islands streak detection, running benchmarks, median calculation in MySQL, counterfactual scenario modelling with sensitivity analysis, rule-based risk segmentation, and — in Section 08 — the habit of testing each query against its own output and documenting what should change.

END OF DOCUMENT